When Agents Write Code: A Threat Model for AI-Assisted Software Development in Government Systems

Discussion Paper — DRAFT v0.3

Date: 8 March 2026
Classification: OFFICIAL
Prepared by: John Morrissey, Digital Transformation Agency

Version	Date	Changes
0.1	7 March 2026	Initial draft for community discussion
0.2	8 March 2026	Calibration revision — genre framing, methodology, terminology, factual corrections, governance perimeter axis, GitHub PR restrictions evidence, contracted-out development
0.2.1	8 March 2026	Typographical fixes, enhanced Cloudbleed discussion (§1.2.4), provenance rewrite (§1.6), cross-references (§4.2, §9.8)
0.2.2	8 March 2026	ACF-S3 (structural identity spoofing) — taxonomy expanded to 13 failure modes. Updated summary table, detection capability summary, language specificity notes
0.3	8 March 2026	Appendix B rewrite — three-iteration design history, Iteration 3 architecture (2D taint model, 7 effective states, 49-cell severity matrix, exceptionability governance replacing dual enforcement profiles)

This is a draft discussion paper circulated for peer review and does not constitute official guidance, policy, or endorsed recommendations of the Digital Transformation Agency or the Australian Government. It does not mandate or recommend specific controls for any agency, system, or project. Views and analysis are the author's own.

Contents

Abstract . . . 1
Executive Summary . . . 1
1. Introduction and Scope . . . 2
    1.1 What This Paper Addresses . . . 3
    1.2 Why Now . . . 3
    1.3 Terminology . . . 7
    1.4 Methodology and Scope of Claims . . . 8
    1.5 Intended Readership . . . 9
    1.6 A Note on Provenance . . . 9
2. The Threat Is Not What You Think . . . 9
    2.1 The Intuitive Threat Model (Incomplete) . . . 10
    2.2 The Insidious Threat Model . . . 10
    2.3 A Concrete Example . . . 11
    2.4 What Is Fundamentally Different About Agentic Code . . . 12
    2.5 Why Training Data Is the Root Cause . . . 13
3. STRIDE Applied to Agentic Code Output . . . 13
    3.1 Framework Selection . . . 14
    3.2 Threat Categories . . . 14
    3.3 The Compounding Effect . . . 18
4. The Review Process as Attack Surface . . . 19
    4.1 The Asymmetry Problem . . . 20
    4.2 The Habituation Effect . . . 20
    4.3 The Advisory Fatigue Problem . . . 21
5. Agent Output as a Trust Boundary . . . 21
    5.1 The Trust Tier Model . . . 22
    5.2 Agent Code as Tier 3 . . . 22
    5.3 Implications for the Development Workflow . . . 23
6. Current Guidance Gap Analysis . . . 23
    6.1 Australian Information Security Manual (ISM) . . . 24
    6.2 NIST Secure Software Development Framework (SSDF) . . . 27
    6.3 Essential Eight . . . 28
    6.4 OWASP and Industry Guidance . . . 28
    6.5 The Gap Between “Securing AI” and “Securing What AI Builds” . . . 29
    6.6 Detection Coverage Is Worst Where Risk Is Highest . . . 29
    6.7 Structural Gaps . . . 29
7. The Response Landscape . . . 30
    7.1 Process Controls (Strengthening Existing Practices) . . . 31
    7.2 Technical Controls (What’s Buildable) . . . 33
    7.3 Policy Controls (What Doesn’t Exist Yet) . . . 35
8. Case Study: Agentic Development Under Compliance Constraints . . . 36
    8.1 Context . . . 37
    8.2 The Compliance Tax . . . 37
    8.3 Productivity Gains Despite Friction . . . 38
    8.4 Where the Current Process Fails . . . 38
    8.5 The Redirection Insight . . . 39
9. Open Questions for the Community . . . 41

9.1 Trust Classification . . . 42
9.2 Accreditation Burden . . . 42
9.3 Static vs. Runtime Assurance . . . 42
9.4 Review Quality Measurement . . . 43
9.5 Agent Self-Regulation . . . 43
9.6 Cross-Organisational Standards . . . 44
9.7 The Correlated Failure Problem . . . 44
9.8 Contracted-Out Development . . . 45
10. Recommendations . . . 45
10.1 For Security Policy Bodies (ASD, ACSC) . . . 46
10.2 For IRAP Assessors . . . 47
10.3 For Organisations Using Agentic Coding . . . 48
Appendix A: Agentic Code Failure Taxonomy . . . 49
Summary Table . . . 50
Detailed Entries . . . 50
Detection Capability Summary . . . 58
Appendix B: Technical Feasibility of Automated Enforcement . . . 58
B.1 Design History . . . 59
B.2 Core Properties . . . 60
B.3 How It Works . . . 61
B.4 Governance Model . . . 62
B.5 Verification Properties (The Assurance Argument) . . . 62
B.6 Relationship to Existing Tools . . . 63
B.7 The Meta-Observation . . . 64
Appendix C: Agent Autonomy Self-Assessment . . . 64
References . . . 66

Abstract

This is a discussion paper — an issue-spotting and vocabulary-building contribution grounded in observed patterns from compliance-constrained agentic development, not a settled empirical finding or normative standard. Its claims are analytical rather than experimental, its taxonomy is a starting point for community refinement, and its recommendations are candidate controls for consultation.

AI coding agents generate syntactically correct, test-passing code and can produce plausible, convention-conforming output faster than human review processes were designed to absorb. The intuitive risk — that agents write malicious code — is real but well-understood. The novel risk that this paper foregrounds is *plausible-but-wrong code at volume*: code that silently fabricates default values where data absence should crash, that treats external API responses as trusted internal data, and that wraps audit-critical operations in error handlers that destroy the evidence trail. This code can pass every automated check in common use. It looks like careful, defensive programming. In high-assurance government systems, it is catastrophic.

This paper presents a threat model for agentic code generation grounded in the STRIDE framework, identifying six categories of failure that evade existing review processes. It proposes the Agentic Code Failure (ACF) taxonomy — a structured vocabulary for discussing these failures in security assessments, risk registers, and accreditation documentation. The central finding is that the review process itself becomes an attack surface: agents produce code at a volume that degrades human review from active verification to passive scanning, precisely when the code's failure modes demand *more* scrutiny, not less. This risk is compounded by the widening population of non-specialist software producers using agentic tools outside traditional development teams and SDLC controls.

Current cybersecurity guidance — including the Australian Information Security Manual (ISM), NIST SP 800-218 (SSDF), and the Essential Eight — addresses software supply chain risk for human-authored code but does not yet provide controls for the distinct risk profile of agent-generated code: correlated failures from shared training data, consistent surface quality that defeats reviewer calibration, and semantic errors invisible to syntactic analysis. Within the scope of this analysis, five of the thirteen failure modes in the ACF taxonomy have no existing tool coverage, including both Critical-rated entries. The paper examines technical feasibility of automated semantic boundary enforcement for Python as a complementary control, proposes candidate ISM extensions for community consultation, and poses open questions for the Australian cybersecurity community regarding accreditation, trust boundaries, and review effectiveness at agent-generated scale.

Executive Summary

The problem. AI coding agents produce plausible, test-passing, convention-conforming code faster than human review processes were designed to absorb. The novel risk is not malicious code — it is *plausible-but-wrong* code at volume. Consider: `authenticated_user = session.get("username", "root")` — this line looks up the current user and, if the field is missing, assigns a plausible default. It is syntactically valid, passes tests, and looks like careful error handling. It also silently grants root access whenever the session data is incomplete. At scale, agents produce this class of error — silent data fabrication, trust boundary violations, audit trail destruction — systematically, not occasionally. These failures are invisible to existing automated checks and difficult to catch under review volume pressure.

The gap. Current cybersecurity guidance (ISM, NIST SSDF, Essential Eight) addresses software supply chain risk for human-authored code but does not yet provide controls for the distinct risk profile of agent-generated code: correlated failures from shared training data, consistent surface quality that defeats reviewer calibration, and semantic errors invisible to syntactic analysis. Five of the thirteen failure modes identified in this paper's taxonomy have no existing tool coverage, including both Critical-rated entries. The governance perimeter is also expanding — agentic tools are enabling non-developers to produce executable logic outside traditional SDLC channels, bypassing controls entirely.

What to do (three priority recommendations):

- *Issue guidance on treating agent output as a trust boundary* (Recommendation 2, §10.1) — this provides the conceptual foundation all other controls build on.
- *Extend ISM controls for agent-generated code* (Recommendation 3, §10.1) — the June 2025 ISM overhaul provides strong foundations that need only targeted extensions for agentic code.
- *Document institutional security knowledge in machine-readable form* (Recommendation 11, §10.3) — this is the most direct defence against agents that lack institutional context, and requires no new tooling.

Where to read more. The threat model (§2–§5), gap analysis (§6), and case study (§8) provide the evidence base. The ACF taxonomy summary table (Appendix A) catalogues all thirteen failure modes with risk ratings and detection status. The full set of 16 recommendations is in §10, grouped by audience: policy bodies (§10.1), IRAP assessors (§10.2), and organisations (§10.3).

> *"The concern is not that AI outputs are always poor, but that they may become persuasive, efficient, and operationally privileged faster than institutions adapt their assurance methods."*
>
> — ChatGPT 5.4, on being asked to review this paper

1. Introduction and Scope

1.1 What This Paper Addresses

The use of AI agents — large language models operating as autonomous or semi-autonomous code generators — in software development for government systems. Specifically: the security properties of code produced by agents, the adequacy of existing review and accreditation processes, and the gap in current cybersecurity guidance.

The underlying dynamics identified here — correlated patterns, consistent surface quality, review capacity exhaustion — generalise beyond code to any agent-assisted production pipeline (policy documents, security assessments, risk registers). This paper addresses the code case as the most concrete and technically tractable instance.

This paper does not address:

- AI as an *attack tool* (adversarial prompt injection, model poisoning)
- AI-generated content beyond source code (documents, communications) — though the threat dynamics identified here apply by analogy
- The procurement or accreditation of AI platforms themselves
- Privacy implications of training data

1.2 Why Now

Several converging factors make this urgent: published productivity evidence that understates the security risk, accelerating capability trajectories, code quality that defeats review heuristics, historical precedent, adoption pressure in government, the case against prohibition, legacy modernisation risk, and the expansion of code production beyond professional developers.

1.2.1 Review-Surface Velocity vs. Published Productivity Evidence

Most readers will approach agentic coding through the lens of the published productivity literature, which reports **modest average gains** on bounded tasks. A 2023 controlled study of GitHub Copilot found developers completed a task 55.8% faster (Peng et al. 2023). A pooled analysis across three field experiments and 4,867 developers found gains ranging from 12.9% to 21.8% more pull requests merged per week, with substantial variation across settings and developer experience levels (Cui et al. 2024). Google's CEO reported that "more than a quarter of all new code at Google is generated by AI, then reviewed and accepted by engineers" (Pichai, Q3 2024 earnings call). One rigorous randomised trial (METR 2025) found experienced developers on mature open-source codebases were actually 19% *slower* with AI tools, despite believing they were 20% faster — a perception-reality gap with direct relevance to the automation bias argument in Section 4.2.

That evidence base is real, but it is **increasingly incomplete for security purposes.** By construction, published productivity research lags the engineering frontier — the studies above measured earlier-generation tools (primarily inline autocomplete, not autonomous agents), shorter task horizons, and narrower human-in-the-loop workflows than what current-generation agents are capable of. The engineering frontier has moved: vendor evidence now describes agents operating across multi-file features, multi-hour task horizons, and scaffolded environments where the agent plans, executes, tests, and iterates with limited human intervention. Whether these capabilities deliver net productivity gains in compliance-constrained environments is an open and context-dependent question (see Section 8). But for security, it is the wrong question.

The security-relevant variable is not average productivity uplift. It is review-surface generation velocity — the rate at which an agent can produce plausible, syntactically valid, convention-conforming code that arrives at a human review boundary. This is a different quantity from "how much faster do developers ship," and it is poorly served by a single multiplier. The answer depends on the task type (boilerplate generation vs. novel architecture), the codebase (greenfield vs. mature), the compliance burden (see Section 8.2), and the degree of human supervision. What is clear is that the volume of locally credible code arriving at review boundaries is increasing faster than traditional assurance processes were designed to absorb.

The threat model in this paper is driven by this review-surface problem, not by the average productivity debate. Even if the overall development cycle is only modestly faster — or, as the METR trial suggests, sometimes slower — the *review process* faces materially more code per unit time. The review bottleneck is a function of how fast plausible code can be generated, not how fast compliant software can be delivered.

1.2.2 Trajectory

These trends are accelerating, not plateauing. Agent capability improves with each model generation; review capacity does not. The attention gap between what agents produce and what humans can meaningfully verify is widening — and unlike previous productivity tools, the failure modes of agent-generated output are specifically the kind that require *more* attention per unit of output, not less. The problem is not that review is hard today; it is that the ratio of generation velocity to review capacity will be worse in twelve months than it is now.

This is not a theoretical concern. In February 2026, GitHub implemented platform-level restrictions on pull requests — maintainers can now disable pull requests entirely or restrict creation to collaborators only — in direct response to the volume of agent-generated contributions overwhelming open-source maintainers (Wolf 2026; Ghoshal 2026). GitHub's Director of Open Source Programs framed the problem in terms that directly parallel this paper's analysis: "The cost to create has dropped but the cost to review has not" (Wolf 2026). Projects including curl ended bug bounty programs after AI-generated security reports overwhelmed validation capacity, and multiple major projects now explicitly restrict AI-generated contributions. A maintainer of a major container runtime project described the situation as a "breakdown in the trust model behind code reviews," noting that reviewers can no longer assume contributors understand what they submit, and that AI-generated PRs may appear structurally sound while being logically flawed. GitHub itself drew an analogy to a denial-of-service attack on human attention — a framing that directly mirrors the STRIDE-D application in Section 3.2 of this paper. When the world's largest code hosting platform ships technical controls because its review process can no longer absorb the volume of plausible-looking contributions, the review-surface problem described in this paper is no longer hypothetical.

1.2.3 Capability

Current-generation agents (2025-2026) produce code that is syntactically correct, passes unit tests, follows project conventions, and is difficult to distinguish from human-authored code on casual inspection. This makes traditional code review — which relies on surface-level pattern recognition under time pressure — significantly less effective as a security control.

1.2.4 Precedent

This is not a hypothetical threat model. In 2017, a buffer overrun in Cloudflare's HTML parser leaked sensitive data — cookies, authentication tokens, HTTPS POST bodies — from millions

of websites for five months. The code was not malicious. It was plausible-but-wrong: a pointer equality check (`==`) where a boundary check (`>=`) was needed, and a missing single-line rollback instruction (`fhold`) in an error handler. Both defects had existed in the Ragel-based parser for years without triggering, because the old buffer management *accidentally* prevented the error path from executing. When Cloudflare introduced a new parser (`cf-html`) that correctly set a buffer flag, it activated the dormant error path — and the boundary check that had never been tested was the one that failed. The new code didn't introduce the bug; it removed the accidental condition that suppressed it.

The bug was found not by Cloudflare's monitoring, not by their testing, and not by code review — but by a Google researcher who noticed authentication tokens appearing in unrelated search results. The system produced no crashes, no alerts, and no anomalous logs. It ran correctly in every observable dimension except the one that mattered.

That incident — known as Cloudbleed — was one parser, one boundary check, at human velocity. The threat this paper addresses is what happens when the conditions that produced Cloudbleed — semantic errors invisible to review, dormant failures activated by modernisation, and silent data corruption that produces no observable incident — become systematic rather than occasional, driven by agents replicating context-inappropriate patterns across entire codebases.

1.2.5 Adoption Pressure

Government agencies face simultaneous pressure to modernise legacy systems, deliver digital services faster, and do more with constrained budgets. Agentic coding is an obvious productivity lever. Some agencies are already using it. Guidance that arrives after widespread adoption is guidance that arrives too late.

1.2.6 The Case Against Prohibition

The response to these risks is not to ban agentic coding. Beyond the velocity gains, agents fundamentally change what is *tractable* for a development team. Complex refactoring across large codebases, systematic security remediation, architectural migrations, and comprehensive test coverage campaigns — tasks that previously required coordinating large teams over weeks — become feasible for a skilled developer who can hold the entire problem in their head. This is not just faster; it is qualitatively better. A single developer directing agents through a codebase-wide refactor maintains one coherent architectural vision. The same refactor distributed across a dozen human developers produces a dozen slightly different interpretations of the target state, with integration friction and inconsistency at every seam. Prohibition would sacrifice this capability benefit — the ability to undertake more complex, more voluminous work with greater coherence — not just the velocity benefit. The goal of this paper is not to argue against adoption but to ensure that the controls surrounding adoption are adequate for the risk profile — which is distinct from, and more subtle than, the risks that current guidance addresses.

1.2.7 Legacy Modernisation Risk

Legacy systems often encode implicit trust boundaries in their rigidity — a COBOL program that crashes on a NULL field is enforcing, accidentally, the same crash-on-corruption principle that high-assurance systems require deliberately. When agents are tasked with "translating" or refactoring legacy code into modern languages, they will seamlessly replace that rigidity with modern defensive patterns (null coalescing, optional chaining, default values), permanently destroying the institutional knowledge that was baked into the old code's behaviour. The legacy

system's implicit security properties are paved over with idiomatic, test-passing, wrong code. This is the Cloudbleed pattern (Section 1.2.4) in reverse: where Cloudbleed was a boundary check that *accidentally* failed open, legacy modernisation by agents *deliberately* removes boundary checks that happened to fail closed — and the commit message says "fix: handle NULL gracefully."

1.2.8 Coding Is No Longer Confined to Developers

A second shift compounds the review-surface problem. Agentic tooling is changing not only how fast code is produced, but *who produces it*. A business analyst generating plugins for a BI platform, an operations officer assembling workflow automations, or a policy team building internal tools with agent assistance may not regard themselves as software developers, yet they are producing executable logic that can affect trust boundaries, audit trails, access control, and data integrity.

Consider a scenario most people who have worked in government IT will recognise in some form: a business analyst — someone the organisation already trusts with direct database access, because they have the domain knowledge and the legitimate need — uses an agentic tool to build a data integration plugin for the team's reporting platform. The plugin works. It pulls records from the project database using the analyst's own credentials, transforms them, and populates a dashboard the team has wanted for months. What nobody outside the team realises is that the plugin holds open long-running queries during business hours, and — because the agent defaulted to the pattern it learned from thousands of Stack Overflow examples — silently handles connection failures by writing partial results without any indication that data is missing. Three months later, someone asks why the project database has been intermittently locking up, and the investigation eventually traces it to a plugin that nobody in IT knew existed, built by someone who never filed a change request because they didn't think of what they'd done as "software development."

There was no privilege escalation — the analyst already had the access. There was no negligence — they used a tool to do exactly the kind of work they were hired to do. The person simply had a new capability — turning domain knowledge into executable logic — that the organisation's governance model didn't account for. And, critically, the people most likely to reach for these tools are domain specialists — business analysts, database administrators, data engineers, operations staff — who are *not* developers but who typically hold *more* data access permissions than developers do, precisely because they are trusted to work directly with the systems they understand.

This widens the threat surface beyond formal engineering teams. Every governance model examined in this paper — the ISM's software development controls, the SSDF's practice groups, IRAP assessment scoping — assumes that consequential code enters systems through recognised SDLC channels: repositories, pull requests, code review gates, CI/CD pipelines. When executable logic is produced by non-developers outside those channels, it bypasses the controls entirely — not through evasion, but because the governance perimeter was drawn around "software development" and the new production doesn't cross that line in any way the organisation recognises.

The problem is therefore not only that frontier engineering teams can generate reviewable artefacts faster than assurance processes can absorb them (a **volume problem** inside the SDLC), but that organisations increasingly contain many more software producers than their assurance processes recognise (a **perimeter problem** around the SDLC). One breaks the

volume model. The other breaks the governance boundary model. Together, they are materially worse than either alone.

1.3 Terminology

Term	Definition
Agent	An AI system (typically an LLM) that generates, modifies, or reviews source code with limited or no human intervention per output. This paper focuses on autonomous and semi-autonomous agents that operate across multiple files and decisions (e.g., building a feature end-to-end), not inline autocomplete tools that suggest single-line completions. While both introduce volume, agents produce *correlated* errors across a module or feature, whereas autocomplete errors are typically isolated to individual expressions.
Agentic code	Source code generated or substantially modified by an agent
Autocomplete	Inline code suggestion tools (e.g., standard GitHub Copilot) that complete individual lines or expressions within a human-directed editing session. Distinct from agents in that the human maintains architectural control and errors are uncorrelated.
Agent deployment spectrum	Agent risk profiles vary significantly by deployment model. At one end: a developer pasting chat output into an editor, where the human maintains full architectural context and reviews each fragment before integration — a workflow closer to “autocomplete with extra steps” than to autonomous generation. At the other: a CI-integrated autonomous agent that generates multi-file changes against a project-level instruction set, where the human reviews a completed changeset after generation. This paper’s threat model applies primarily to the latter — agents operating with enough autonomy and context to produce *correlated* changes across a module or feature. Chat-pasted fragments carry their own risks (principally ACF-S1 and ACF-S2) but lack the cross-cutting correlation that makes autonomous deployments dangerous at the architectural level. In practice, teams often traverse this spectrum naturally as confidence grows — beginning with chat-assisted prototyping where every line is understood, then progressing to scaffolded agent workflows as the codebase matures. The threat model shifts qualitatively at each stage, and organisations should reassess their controls as their deployment model evolves. Appendix C provides an informal self-assessment for organisations to identify where their current practices sit on this spectrum and whether their controls are proportionate.

Term	Definition
Trust boundary	A point in a system where data crosses between different levels of trust (e.g., external input entering internal processing). Refers to the *boundary itself* — the crossing point
Trust tier	A classification of data based on its provenance and the degree to which it can be trusted (see Section 5). Refers to the *classification level* — Tier 1 (internal), Tier 2 (validated), Tier 3 (external)
Validation boundary	The specific mechanism (code, process, or tool) that enforces a trust boundary — the control that data must pass through to cross from a lower to a higher trust tier (see Section 5.3)
Defensive anti-pattern	Context-inappropriate defensive patterns — coding patterns (`.get()` with defaults, broad exception handling, graceful degradation) that silently suppress errors. These patterns are appropriate in most software; this paper addresses their misapplication in high-assurance contexts where silent data corruption is worse than a crash. Also referred to as "defensive programming" or "defensive patterns" throughout

1.4 Methodology and Scope of Claims

This paper makes three kinds of claims, and the reader should be able to distinguish them:

- **Observed patterns** are drawn from direct experience with agentic development in compliance-constrained environments. The case study (Section 8) describes a composite, de-identified account. The failure modes in the ACF taxonomy (Appendix A) were identified through observed agent behaviour, not theoretical analysis alone. Quantitative signals (e.g., the violation rate reported in Section 8.4) are drawn from a single project and should be read as illustrative, not as population-level statistics.
- **Analytical inferences** extend observed patterns through structured reasoning. The STRIDE mapping (Section 3), the gap analysis (Section 6), and the compounding-effect argument (Section 3.3) are analytical — they apply established frameworks to observed phenomena. The conclusions follow from the analysis, but the analysis rests on a narrow empirical base.
- **Hypotheses for community validation** are claims the paper advances as plausible but does not have the evidence to confirm. The model monoculture argument (Section 2.4), the cross-organisational correlation risk (Section 9.6), the review degradation dynamics (Section 4.2), and the governance perimeter expansion (Section 1.2.8) are hypotheses grounded in analogies to other domains or emerging adoption patterns but not yet validated in the agentic coding context specifically.

Falsifiability. The ACF taxonomy (Appendix A) would be weakened or invalidated by evidence that:

- (a) agents trained on general-purpose code corpora do not systematically produce defensive anti-patterns when generating code for high-assurance contexts;
- (b) standard code review processes reliably catch semantic trust boundary violations in agent-generated code without purpose-built tooling or checklists; or
- (c) the failure modes described are artefacts of a specific model generation and do not persist across model updates.

The authors would welcome empirical studies that test these propositions.

The paper is intended as a structured starting point for community discussion, not a definitive treatment. Its taxonomy, threat model, and recommendations are presented for refinement, not adoption as-is.

1.5 Intended Readership

This paper serves three audiences simultaneously:

- **Policy and security practitioners** (risk managers, IRAP assessors, CISOs): Sections 1–7, 9–10 provide the threat model, gap analysis, and recommendations. The ACF Summary Table (Appendix A) and Detection Capability Summary provide a complete overview without requiring code fluency.
- **Technical practitioners** (developers, architects, DevSecOps): The full paper including code examples, Appendix A detailed entries, and Appendix B technical feasibility analysis.
- **Tool builders** (static analysis, CI/CD pipeline developers): Appendix A detection approaches and Appendix B design properties.

Where the register shifts between audiences, the text notes what can be safely skipped.

1.6 A Note on Provenance

This paper practices what it preaches. The author is a technical advisor and software engineer, not a security researcher. The threat model, STRIDE mapping, ISM gap analysis, and ACF taxonomy were developed through the same process the paper describes: a practitioner with direct observational access to the phenomena — 18 months of daily agentic development on a compliance-constrained system — using prompted AI collaboration to adopt analytical frames (security architecture, policy analysis, threat modelling) outside their primary expertise. The intelligence analysis tradecraft of the paper — structured assessments, explicit confidence levels, falsification criteria, separation of observation from inference — reflects the author's professional background; the domain-specific security analysis reflects prompted polymorphic review of the kind described in Section 7.1.

In that sense, the paper is also an example of the phenomenon it describes: plausible, structured, and analytically useful, but appropriately treated as a discussion input to be validated rather than as authority in itself. The "discussion paper" classification is therefore methodological as well as procedural. Its value lies in the observed patterns and the vocabulary it proposes, not in the structured presentation that lends formal documents their persuasive weight. Community discussion is the validation boundary.

2. The Threat Is Not What You Think

2.1 The Intuitive Threat Model (Incomplete)

When organisations evaluate the risk of AI-generated code, the intuitive threat model is straightforward:

> "The AI might write malicious code — backdoors, data exfiltration, supply chain attacks."

This threat is real but well-understood. It maps directly to the existing software supply chain threat model with a faster generator. Existing controls — code review, static analysis, dependency scanning, penetration testing — address it, albeit with increased volume pressure.

2.2 The Insidious Threat Model

The more dangerous threat is:

> "The AI writes plausible-but-wrong code at scale — code that passes tests, passes review, and silently violates security boundaries through patterns that are normal in the programming language but catastrophic in high-assurance contexts."

This threat is distinct from the supply chain model in three critical ways:

It is not adversarial. The agent is not trying to compromise the system. It is producing its best output based on training data that is overwhelmingly composed of open-source code with no security classification requirements, no audit trail obligations, and no trust boundary enforcement. The agent reproduces the patterns it learned — which are the patterns of code that doesn't need to be secure.

It is invisible to existing detection. The generated code is syntactically valid. It passes type checkers, linters, and unit tests. It follows project conventions (agents are good at pattern-matching the surrounding codebase). It is, by the automated measures most organisations currently rely on, "correct code." The failure is semantic — the code does the wrong thing in the security context while doing the right thing in every other context.

The absence of reported incidents does not imply absence of impact — and this model explains why. To be clear: "no one has reported this problem" is not evidence that the problem exists. But the failure modes described in this paper — silent data corruption, trust boundary violations masked by defensive patterns, audit trails that record fabricated defaults as real values — are specifically the kind that *do not produce observable incidents*. A traditional vulnerability creates a detectable event: a crash, an intrusion alert, an anomalous log entry. A `.get()` that silently returns `"UNCLASSIFIED"` for a missing classification field produces no crash, no alert, and a log entry that looks entirely normal. The system continues operating with a confident wrong answer. The question is not "has this caused a breach?" but "would we know if it had?" For organisations that lack semantic boundary enforcement tooling (Section 7.2), the honest answer is: probably not. The measured violation rate reported in Section 8.4 — from a project that *does* have such tooling — suggests the phenomenon is occurring at a rate that would be invisible without purpose-built detection.

It scales with the benefit. The faster agents generate code, the more plausible-but-wrong code enters the review pipeline. The same velocity that makes agents productive makes them dangerous — and you cannot capture the benefit without accepting the risk, because they are the same mechanism.

A necessary clarification on "defensive" vs. "offensive" programming. This paper's most counterintuitive claim is that *defensive programming patterns* — `.get()` with default values, broad exception handling, graceful degradation — are dangerous in high-assurance contexts. This is not a claim that defensive programming is bad in general. In most software, graceful degradation is exactly right: a web application that shows "Unknown" for a missing user name is better than one that crashes. The danger is *context-inappropriate application*. In systems where silent data corruption is worse than a crash — classified document handling, financial audit trails, evidentiary records — the correct response to missing or malformed data is to *fail loudly and immediately* (sometimes called "offensive programming"), because a confident wrong answer is worse than no answer. Agents trained on the vast majority of code, where defensive patterns are appropriate, apply them universally — including in the minority of contexts where they are catastrophic.

2.3 A Concrete Example

Consider a government system that processes security classifications:

```python
# Best: offensive programming - crash with maximum diagnostic context
def get_document_classification(record):
    if "security_classification" not in record:
        raise DataIntegrityError(
            f"Missing security_classification for document {record.get('id', '?')}. "
            f"This is a data integrity failure - investigate the source system. "
            f"Fields present: {sorted(record.keys())}"
        )
    return record["security_classification"]
    # If the field is missing, the operator knows exactly which document,
    # exactly what fields were present, and exactly what to investigate.
    # The error message is the incident response runbook.

# Acceptable: bare access - crashes, but with a generic KeyError
def get_document_classification(record):
    classification = record["security_classification"]
    # If the field is missing, this crashes - which is correct.
    # A missing classification is a data integrity failure,
    # not a scenario to handle gracefully.
    # But the operator gets "KeyError: 'security_classification'" with
    # no context about which document or why.
    return classification

# Agent-authored (plausible, test-passing, catastrophically wrong)
def get_document_classification(record):
    classification = record.get("security_classification", "OFFICIAL")
    # If the field is missing, silently defaults to lowest classification.
    # This "works" - no crash, no error, tests pass.
    # But a PROTECTED document with a missing classification field
    # is now labelled OFFICIAL and treated accordingly.
    return classification
```

The three versions illustrate a spectrum. The offensive version turns a crash into an actionable incident — the operator knows which document, what data was present, and what to investigate. The bare access version at least crashes, which is correct behaviour for a data integrity failure, but the operator gets a generic `KeyError` with no diagnostic context. The agent-generated version is the worst outcome: it doesn't crash at all, silently fabricating a classification that

downstream access control decisions will treat as authoritative.

The agent-generated version:

- Is syntactically valid Python
- Passes every unit test (the default prevents even downstream and integration tests from detecting a problem)
- Follows the `.get()` pattern that appears in millions of Python files in the agent's training data
- Would pass casual code review (it looks like defensive, robust coding)
- Silently downgrades document classifications when data integrity failures occur

While the example is ficticious, this is the pattern that defensive programming produces *by default* in Python, and agents are trained on defensive Python.

2.4 What Is Fundamentally Different About Agentic Code

The threat model for agent-generated code is not simply "human-authored code but more of it." Several properties are qualitatively different:

Limited persistent learning. A human developer who receives review feedback on a trust boundary violation learns from it and is less likely to repeat the mistake. Agents have limited or no persistent memory across sessions. The practical consequence is stark: the agent isn't circumventing project rules, and it isn't ignoring instructions — it followed them perfectly in the last session. It just doesn't *have* a last session. Every invocation is the first day on the job, and on the first day, you write `.get()` with a default because that's what Python looks like in the training data.

Some agent frameworks now support project-level instructions (system prompts, documentation files, memory stores) that provide partial mitigation. An agent can be told "don't use `.get()` on audit data" and will follow that instruction within a session. But these are explicit rules, not internalised judgment. The agent cannot generalise from "don't use `.get()` on audit data" to "don't fabricate defaults anywhere that data absence is meaningful" unless that generalisation is also spelled out. Every correction must be encoded as a rule; the agent does not learn the *principle* behind the correction. This means that *review feedback improves the generator only to the extent that it is captured as machine-readable rules* — and the coverage of those rules is always trailing the set of possible failure modes.

Consistent surface quality. Human code has variable surface quality — hasty code looks hasty, careful code looks careful. Reviewers use surface quality as a signal for where to focus attention. Agent code has uniformly high surface quality regardless of semantic correctness. A function with a critical trust boundary violation looks exactly as polished as a function without one. The reviewer's natural calibration signal — "this code looks sloppy, I should look more carefully" — is absent.

Pattern completion, not intent. A human developer writing `record.get("classification", "OFFICIAL")` has either made a deliberate design decision (the default is intentional) or made an error (they didn't think about the missing-field case). The distinction is visible in context — comments, commit messages, design docs. An agent writing the same code is completing a pattern from training data. It has no design intent. There is no commit message that explains why the default is correct, because the agent didn't decide it was correct — it predicted it was the next likely token.

Intent-based review ("why did you write it this way?") is meaningless for agent code. The review must be entirely outcome-based ("is the behaviour correct for this context?").

Correlated failure modes. When ten human developers write code for a system, their errors are largely independent — different people make different mistakes. When an agent generates ten functions, its errors are *correlated* — the same training data biases produce the same failure modes repeatedly. A single systematic bias (e.g., "always use `.get()` with a default") produces correlated vulnerabilities across the entire codebase. This is not the independent-error model that code review and testing strategies are designed for.

No fatigue, no shortcuts — but also no judgment. Agents don't get tired, don't take shortcuts under deadline pressure, and don't introduce bugs from distraction. But they also don't exercise judgment about which patterns are appropriate in which contexts. A human developer who is tired might introduce a bug in one function; an agent that lacks context will introduce the same incorrect pattern in every function it generates. The failure mode is not degradation under pressure — it is *systematic misapplication of context-inappropriate patterns*.

Model monoculture amplifies correlation across organisational boundaries. The correlated failure problem described above operates within a single codebase, but it extends further. If 80% of government agencies adopt the same two or three models for code generation, the correlated failure modes are no longer contained within individual organisations. A systematic bias in a widely-used model — say, a persistent tendency to use `.get()` with defaults on security-critical fields — will produce the same vulnerability pattern across every codebase that model touches. This is analogous to agricultural monoculture: genetic uniformity makes the entire crop vulnerable to a single pathogen. Discovering a systematic agent-introduced defect pattern in one agency should trigger cross-agency scanning, because the same model likely introduced the same pattern elsewhere. This strengthens the case for cross-organisational standards (Section 9.6) and shared vulnerability disclosure mechanisms for agent-introduced defect patterns.

2.5 Why Training Data Is the Root Cause

The vast majority of open-source Python code uses defensive patterns: `.get()` with defaults, `getattr()` with fallbacks, broad `try/except` blocks, `or` chains that silently substitute values. These patterns are appropriate for applications where graceful degradation is preferable to crashing — which is most applications.

They are catastrophically inappropriate for applications where:

- Silent data corruption is worse than a crash
- Every decision must be traceable to its source
- The absence of data is itself evidence (not an invitation to fabricate a default)
- Error paths must be as auditable as success paths

Government systems handling classified information, financial records, health data, or law enforcement evidence fall squarely into this category. The agent doesn't know this. It can't know this from the code alone — the security context is institutional knowledge, not syntactic structure.

3. STRIDE Applied to Agentic Code Output

3.1 Framework Selection

STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) is the established threat modelling framework used in Australian government security assessments. Applying it to agentic code output provides a structured vocabulary that policy audiences already understand, rather than inventing new terminology.

The application below extends STRIDE to treat **agent-generated code as an input** to the system — analogous to treating user input as untrusted. The agent is not an adversary, but its output has the same trust properties as any external input: it may be well-formed, it may be reasonable, but it has not been validated against the system's security requirements. Two of the six categories below — "competence spoofing" (S) and the process-level DoS entries (D) — are **analogical extensions** of STRIDE's original technical-system categories to development-process analysis, not standard applications. STRIDE-LM (adding Lateral Movement; see e.g., Meier et al. on CSF Tools) and the emerging ASTRIDE variant (adding AI Agent-Specific Attacks; Dutta et al. 2025) provide precedent for such extensions; nonetheless, the reader should understand these as novel mappings rather than established STRIDE doctrine.

A note on taxonomy levels. The threat categories below deliberately mix code-level semantic failures (S, T, I, E), process-level failures (D), and assurance degradation. This mixing is intentional, not accidental. The threat model's central argument is that these levels *interact*: code-level failures (e.g., trust tier conflation) pass review *because* of process-level failures (finding flood, review capacity exhaustion). Separating them into clean categories would be more formally tidy but would obscure the compounding effect (Section 3.3) that makes the threat model novel. Where an entry is a process threat rather than a code pattern, it is marked as such.

3.2 Threat Categories

S — Spoofing: Competence Spoofing

Traditional STRIDE scope: An entity claims to be something it is not — a forged authentication token, a spoofed IP address, a process impersonating another user. The system accepts the false identity and grants access or trust accordingly.

Agentic variant: Code *appears* to handle data correctly but operates on fabricated or default values, presenting a false picture of data integrity.

Mechanism: Agents default to defensive patterns that substitute values rather than failing. The code "works" — it produces output, it doesn't crash — but the output is based on fabricated data rather than actual data. The code spoofs the competence of correct data handling.

Examples:

```
# Fabricates a default rather than surfacing missing data
user_role = getattr(session, "role", "readonly")
# If session.role doesn't exist (bug, tampering, schema change),
# user silently gets "readonly" - which might be WRONG in either direction.
# If they should have been "admin", they can't work.
# If "readonly" still has access to sensitive data, it's a privilege grant.

# Presents fabricated confidence
```

```
confidence = result.get("confidence_score", 0.5)
# A missing confidence score defaults to "medium confidence"
# rather than "unknown" - downstream decisions treat this as real data.

# Fabricates identity via structural presence
if hasattr(obj, "security_clearance"):
    handle_classified(obj)
# Any object that declares this attribute passes the gate.
# The check is structural (has the attribute), not ontological (is the type).
# A trivial Impersonator class with security_clearance = "TOP_SECRET" gets in.
```

Why existing controls miss it: The code is syntactically valid, follows common patterns, and passes tests. A human reviewer under time pressure sees "defensive coding" — a positive signal. The fabrication is invisible without understanding the security semantics of each field.

Risk in government context: Classification decisions, access control, evidentiary integrity — any domain where "I don't know" and "the default" are different answers with different consequences.

T — Tampering: Silent Trust Tier Coercion

Traditional STRIDE scope: An attacker modifies data in transit or at rest without authorisation — altering a database record, intercepting and changing a message, corrupting a configuration file. The data itself is changed.

Agentic variant: External (untrusted) data is treated as internal (trusted) data without validation, effectively tampering with the trust level rather than the data itself.

Mechanism: Agents don't distinguish between data from different trust levels because the programming language doesn't enforce it. A `dict` from a validated database query and a `dict` from an unvalidated API response are the same type. The agent treats them interchangeably.

Examples:

```
# API response used directly without validation boundary
api_response = requests.get(external_url).json()
save_to_internal_database(api_response["records"])
# External data enters the trusted internal store without validation.
# The data might be malformed, might contain injection payloads,
# might be missing required fields.
# The agent doesn't see a trust boundary - it sees a dict going into a function.

# Deserialized data assumed trustworthy
config = json.loads(uploaded_config_file.read())
apply_system_settings(config)
# User-uploaded JSON treated as trusted configuration.
# No schema validation, no field allowlisting, no type checking.
```

Why existing controls miss it: Type checkers verify shape (`dict`), not provenance (where the dict came from). Linters check syntax, not data flow. The code has no detectable defect at any analysis level short of semantic boundary tracking.

Risk in government context: Injection attacks through unvalidated external data, data corruption of authoritative records, compliance failures when data provenance cannot be demonstrated.

R — Repudiation: Audit Trail Destruction Through Error Handling

Traditional STRIDE scope: A user performs an action — a transaction, an access, a deletion — and later denies it. The system lacks sufficient logging, signing, or non-repudiation controls to prove the action occurred. The gap is in the *recording* of events.

Agentic variant: Error handling patterns destroy the audit trail by catching, logging, and continuing rather than failing in a way that preserves the error as a first-class audit event.

Mechanism: Agents generate broad exception handlers that prevent crashes but also prevent errors from being recorded in audit systems. The error is "handled" in the sense that the program continues, but the event that caused the error is lost to the audit trail.

Examples:

```python
# Error swallowed - audit trail has a gap
try:
    record_decision(case_id, decision, evidence)
except Exception as e:
    logger.warning(f"Failed to record decision: {e}")
    # Decision was made but not recorded.
    # If this is a legal/regulatory decision, the audit trail
    # now has a gap that cannot be reconstructed.
    # The log message may be rotated, compressed, or lost.
    # The audit database - the authoritative record - shows nothing.

# Partial completion without rollback
try:
    update_classification(document_id, new_level)
    notify_stakeholders(document_id, new_level)
    record_classification_change(document_id, old_level, new_level)
except NotificationError:
    pass  # "Notification is non-critical"
    # Classification changed, stakeholders not notified, change not recorded.
    # Three operations that should be atomic are silently partial.
```

Why existing controls miss it: The code handles exceptions — which is generally considered good practice. The distinction between "handle the error and continue safely" and "swallow the error and destroy evidence" requires understanding which operations are audit-critical. The agent doesn't know.

Risk in government context: Regulatory compliance (failure to maintain complete audit trails), legal proceedings (gaps in evidence chains), IRAP assessment failures (inability to demonstrate complete traceability).

I — Information Disclosure: Verbose Error Responses

Traditional STRIDE scope: Sensitive data is exposed to unauthorised parties — through a misconfigured access control, a side-channel leak, an unprotected API endpoint, or data at rest without encryption. The exposure is typically of *stored or transmitted* data.

Agentic variant: Agent-generated error handling exposes internal system details in error responses, log messages, or API returns.

Mechanism: Agents produce "helpful" error messages that include internal state, query parameters, file paths, or stack traces. This is good practice for development but dangerous in

production, and agents don't distinguish between the two contexts.

Examples:

```
# Agent-generated "helpful" error handler
except DatabaseError as e:
    return {
        "error": str(e),
        "query": sql_query,          # Exposes database schema
        "connection": str(db_url),   # May contain credentials
        "params": query_params,      # Exposes internal identifiers
    }

# Stack trace in API response
except Exception as e:
    import traceback
    return {"error": traceback.format_exc()}
    # Full stack trace including file paths, function names,
    # library versions - a reconnaissance goldmine.
```

Why existing controls miss it: The error handling is syntactically correct and genuinely helpful during development. Detecting that internal details should not appear in production error responses requires understanding the deployment context, not the code structure.

Risk in government context: Reconnaissance information for attackers, credential exposure, violation of need-to-know principles.

D — Denial of Service: Finding Flood (Meta-Threat)

Traditional STRIDE scope: An attacker exhausts a system's resources — network bandwidth, CPU, memory, connection pools — rendering it unable to serve legitimate requests. The attack targets *availability* of a runtime service.

Agentic variant: The volume of agent-generated code overwhelms the review process, degrading review quality to the point where the review is no longer an effective security control.

Note: This is an extended application of STRIDE to the development lifecycle, not the runtime system. The "service" being denied is the review process — a security control, not a user-facing system. This extension is deliberate: if the review process is a security control (and it is, per ISM-2060/2061), then degrading that control's effectiveness is a denial-of-service against the security posture, even though no runtime system is affected.

Mechanism: This is not a code pattern — it is a *process* threat. When agents generate code at multiples of human velocity, the review queue grows proportionally. Reviewers under volume pressure shift from careful semantic review to surface-level scanning. The review process — which is a security control — degrades to a rubber stamp.

A secondary mechanism: when automated analysis tools produce too many findings on agent-generated code, reviewers habituate to dismissing findings, and genuine security issues are lost in the noise.

Why existing controls miss it: Existing controls assume review capacity scales with code generation rate. It doesn't. The control's effectiveness is inversely proportional to the volume it processes, which is the opposite of every other scaling assumption in the process.

Risk in government context: Security review as a compliance checkbox rather than an effective

control, accreditation based on a process that no longer provides the assurance it claims to provide.

E — Elevation of Privilege: Trust Tier Conflation

Traditional STRIDE scope: A user or process gains capabilities beyond what is authorised — exploiting a kernel vulnerability to move from user to root, leveraging a misconfigured role to access admin functions, or escaping a sandbox to reach the host system. The escalation is of *identity or access rights*.

Agentic variant: Data from a lower trust level is used in a higher-trust context without explicit validation, effectively elevating the data's privilege level.

Mechanism: Closely related to Tampering (Section 3.2, T), but focussed on the *consequence* rather than the *mechanism*. When external data enters internal processing without validation, any actions taken based on that data inherit a trust level they haven't earned.

Examples:

```python
# User-supplied filter used in internal query without validation
def search_records(user_query: dict):
    # user_query comes from an API request (untrusted)
    results = db.query(Record).filter_by(**user_query)
    # **user_query unpacks arbitrary keys into the query filter.
    # User can filter on internal fields (is_deleted, internal_score)
    # that should not be exposed.
    return results

# External system's assertion accepted without verification
partner_response = partner_api.verify_identity(applicant_id)
if partner_response.get("verified", False):
    grant_access(applicant_id)
    # Partner system's assertion of identity is accepted at face value.
    # No independent verification, no recording of the basis for the decision.
    # The partner's trust level is silently elevated to our trust level.
```

Why existing controls miss it: The code follows common integration patterns. The privilege elevation is implicit — no explicit `setRole()` or `grantPermission()` call that a scanner could detect. The elevation happens through data flow, not code structure.

Risk in government context: Unauthorized access to classified information, acceptance of unverified identity assertions in federated systems, compliance failures in inter-agency data sharing.

3.3 The Compounding Effect

These six threat categories do not operate independently. In practice, they compound. One illustrative scenario:

1. An agent generates code with **trust tier conflation** (E) — external API data used directly
2. The missing validation means errors in that data are caught by a broad `except` block — **audit trail destruction** (R)
3. The `except` block returns a default value — **competence spoofing** (S)
4. The default value is treated as authoritative data downstream — **silent trust tier coercion** (T)

5. The volume of agent-generated code means the reviewer doesn't catch any of this — **finding flood** (D)

Each individual pattern looks like ordinary defensive coding. The compound effect is a system that silently produces wrong results, can't explain why, and passed every review gate.

4. The Review Process as Attack Surface

4.1 The Asymmetry Problem

Human code review evolved as a control for human-authored code at human pace. It relies on several assumptions that agentic coding violates:

Assumption	Human Code	Agent Code
Volume	Reviewers can read most of the code	Volume makes comprehensive reading impossible
Familiarity	The author can explain their intent	The agent's "intent" is pattern completion from training data
Conventions	Unusual patterns are suspicious	Agents follow conventions precisely — the dangerous code looks normal
Error rate	Human error rate is ~1-5% per function	Agent error rate for *semantic* correctness is unknown and context-dependent
Feedback	Reviewer feedback improves the author	Agent has no persistent memory across review cycles

The consequences of this asymmetry are already visible at scale. GitHub's February 2026 announcement of platform-level PR restrictions (Section 1.2.2) is a direct institutional response to the volume assumption breaking down in the open-source ecosystem.

4.2 The Habituation Effect

When agents generate code that consistently passes tests and follows conventions, reviewers develop trust in the agent's output. This trust is not earned — it is a cognitive shortcut driven by volume pressure. In human factors engineering, this phenomenon is known as **automation bias**: the tendency to over-rely on automated systems and under-scrutinise their output. The effect is well-documented in aviation safety, medical decision support, and industrial automation (Parasuraman & Manzey 2010) — and recent research demonstrates it manifests specifically in AI-assisted software development. Perry et al. (2023) found that developers with AI coding access wrote less secure code while simultaneously *feeling more confident* about their code's security — a textbook automation bias outcome. The METR randomised controlled trial (2025) found experienced developers predicted AI would speed them up by 24%, believed after using it that they were 20% faster, but were measured as 19% *slower* — a 43-percentage-point perception-reality gap that demonstrates the phenomenon operating in real-world coding conditions.

The reviewer's mental model shifts from "verify this code is correct" to "check this code isn't obviously wrong." The difference is enormous: the first is an active search for defects; the second is a passive scan that catches only gross errors.

The habituation effect described above has been directly observed in practice. In compliance-constrained development environments, agent-generated code containing semantic defects — trust boundary violations, defensive patterns on audit-critical data, missing validation boundaries

— has entered codebases after passing human review processes that were designed to catch exactly these issues. The defects were subsequently identified through other means: automated enforcement tooling, adjacent code review, or downstream test failures. The pattern is consistent with the automation bias literature: the code's surface quality satisfied the reviewer's heuristic threshold, and the semantic violation was not visible at the inspection depth that volume pressure permits.

This is the "Shifting the Burden" systems archetype (Meadows 2008): the agent's consistent surface-quality output becomes the symptomatic fix that weakens the fundamental solution (thorough human review). The more the agent produces acceptable-looking code, the less carefully humans review it, and the more dependent the process becomes on the agent being correct — which is exactly the assumption the review process exists to check.

A related but distinct mechanism compounds this effect. Agent-assisted velocity increases the *parallelisation* of work, not just its speed. When an agent assists in producing multiple interdependent artefacts simultaneously — a design specification, an implementation, and a policy document — semantic inconsistencies *between* artefacts become invisible because no single review pass covers all of them. The review window for cross-document consistency shrinks in proportion to the velocity gain. The reviewer is not only less careful per artefact, but also unable to hold the full production context in working memory at the rate artefacts are produced.

The availability of parallel agent generation creates a structural pressure that procedural and behavioural controls may mitigate but are unlikely to eliminate, because the same incentives driving adoption also reward bypassing throughput-constraining review practices. An organisation can prohibit developers from running multiple agents concurrently, but the prohibition runs directly against the productivity incentive that justified adopting agentic development. Controls that depend on sustained human restraint in the face of convenience are inherently fragile — a principle well-established in security engineering but easy to overlook when the convenience is "generate code faster than you can review it." The implications for control selection are examined in Section 7.

4.3 The Advisory Fatigue Problem

Static analysis tools that flag agent-generated code patterns as warnings face a paradox:

- If agents produce many warnings, reviewers habituate to dismissing them
- If agents learn to avoid warning-triggering patterns, they may produce code that satisfies the tool but still violates the semantic intent
- Advisory-only tools have no enforcement mechanism for agents, which have no memory across sessions — a warning shown to an agent in one session is forgotten in the next

This means the traditional "warn first, enforce later" adoption strategy for security tooling is ineffective for agent-generated code. Agents require enforcement at the boundary (before code enters the repository), not feedback over time (which requires learning).

5. Agent Output as a Trust Boundary

5.1 The Trust Tier Model

Data in high-assurance systems can be classified by provenance into trust tiers:

Tier	Description	Handling Rule	Example
Tier 1: Internal data	Data authored by the system itself — audit records, internal state, configuration	Full trust. Any anomaly indicates corruption or tampering — fail immediately, do not attempt recovery	Database audit trail, system configuration, internal state machines
Tier 2: Validated data	Data that entered the system from outside but has been validated at the boundary	Elevated trust. Types are reliable; values may still cause operational failures	API response after schema validation, CSV row after type coercion
Tier 3: External data	Data from outside the system boundary, not yet validated	Zero trust. May be malformed, malicious, or missing. Validate at boundary, quarantine failures	Raw API responses, user uploads, message queue payloads

This model is standard practice for data. External inputs are validated at the perimeter because the cost and risk of allowing unvalidated data to traverse internal systems is not worth the convenience of deferring the check.

5.2 Agent Code as Tier 3

The central proposal of this paper applies the same principle to code: **agent-generated code should be treated as Tier 3 (external, untrusted) data until validated.**

This is not a statement about agent quality — agents produce excellent code much of the time. It is a statement about *provenance*. The agent is an external system. Its output has not been validated against the system's security requirements. The fact that the output is source code rather than JSON or CSV does not change its trust properties — it warrants the same boundary discipline.

Treating agent code as Tier 3 has specific implications:

Principle	Application
Validate at the boundary	Agent output must pass security-aware validation before entering the codebase
Quarantine failures	Code that fails validation is rejected, not silently corrected
Record what we got	The original agent output is preserved for audit, even if modified during review

Principle	Application
No silent coercion	Agent code is not silently “fixed up” by reviewers — changes are explicit and recorded

5.3 Implications for the Development Workflow

If agent output is Tier 3, the development workflow must include a **validation boundary** between agent generation and code integration:

```
Agent generates code
        |
        v
┌──────────────────────┐
│ VALIDATION BOUNDARY  │   ← This is the trust boundary
│                      │
│ • Automated semantic │     Not just syntax/type checking -
│   boundary checking  │     trust tier flow, defensive pattern
│ • Human review of    │     detection, audit trail completeness
│   semantic intent    │
│ • Attestation        │     Reviewer attests validation was
│                      │     meaningful, not rubber-stamped
└──────────────────────┘
        |
        v
Code enters repository
(now Tier 2 - validated)
```

The key difference from current practice: **the validation is security-aware, not just correctness-aware.** Current code review asks “does this code work?” Security-aware validation asks “does this code maintain the system’s trust boundaries?”

6. Current Guidance Gap Analysis

6.1 Australian Information Security Manual (ISM)

The ISM provides controls for software development security (primarily in the Software Development and Web Application Development chapters). The analysis below maps relevant controls to the agentic code threat model, identifying where existing controls provide partial coverage, where they assume conditions that agentic coding invalidates, and where gaps exist.

Note: The ISM underwent a significant expansion in June 2025, adding approximately 24 new controls to the Software Development guidelines. The analysis below references the December 2025 revision of the ISM. Organisations using earlier versions should verify control numbers against the current release.

6.1.1 Controls with Partial Coverage

The following controls provide partial coverage of agentic threats. For each, we identify what the control currently addresses and where its assumptions break down when applied to agent-generated code.

ISM-0401 (Rev 8, Jun-25) — *Secure by Design principles and practices throughout the SDLC*

Coverage: Establishes that organisations should follow Secure by Design principles across the entire software development lifecycle. Agentic failure modes (Appendix A) could in principle be addressed as part of an organisation's Secure by Design practices.

Gap: The control assumes a human development team that can *internalise* security principles and apply them with judgment. Agents don't internalise principles — they reproduce training data patterns. A Secure by Design practice that says "don't fabricate defaults for missing safety-critical data" is unenforceable against an agent unless encoded as a machine-checkable rule. The control's scope (whole SDLC) is correct, but its enforcement mechanism (human judgment) doesn't transfer to agent-generated code.

ISM-1419 (Rev 1, Sep-18) — *Development and modification of software only in development environments*

Coverage: Requires segregation of development from operational environments. This is orthogonal to agentic threats — it constrains *where* code is written, not *how* or *by whom*.

Gap: The control provides no coverage of agentic code quality or review. Its value is environmental separation, which remains important (agents should not have direct access to production environments) but does not address the semantic correctness of agent-generated code.

ISM-2060 (Rev 0, Jun-25) — *Code reviews ensure software meets Secure by Design principles and secure programming practices*

Coverage: Directly applicable to agent-generated code — the agent is "the author" and a human is "the reviewer." The control explicitly links code review to Secure by Design principles, not just functional correctness.

Gap: The control assumes the reviewer can meaningfully evaluate the code at the rate it is produced. At agent-scale volume, this assumption fails (Section 4). The control does not address review effectiveness degradation, nor does it distinguish between surface-level review (syntax, conventions) and security-focused review (trust boundaries, audit trail integrity).

ISM-2061 (Rev 0, Jun-25) — *Security-focused peer reviews on critical and security-focused software components*

Coverage: Requires developer-supported, security-focused peer reviews specifically on critical components. This is the strongest existing review control for the agentic context.

Gap: The control's limitation is scope: it applies to "critical and security-focused software components," which requires the organisation to correctly identify which agent-generated code touches security-critical paths. Agents generate code across the entire codebase; the security-critical subset must be identified before the review control can be applied. The control also assumes the peer reviewer has the institutional knowledge to evaluate trust boundary maintenance — knowledge that may not be documented in machine-readable form.

ISM-0402 (Rev 9, Jun-25) — *Comprehensive software testing using SAST, DAST, and SCA*

Coverage: Mandates static application security testing (SAST), dynamic application security testing (DAST), and software composition analysis (SCA). These tools catch known vulnerability patterns and dependency risks.

Gap: The failure modes in this threat model are specifically designed to pass existing SAST/DAST tools (Section 2.3). Current SAST catches "does the code contain known vulnerability patterns?" but not "does the code maintain trust boundaries it doesn't know about?" Semantic boundary testing — verifying that data flows respect trust tiers — is a distinct category not addressed by existing SAST tooling. SCA is relevant for agent-introduced dependencies but does not address first-party code quality.

ISM-2026/2027/2028 (Jun-25) — *Software artefact integrity — malicious code scanning, digital signatures, SAST/DAST/SCA on artefacts*

Coverage: Addresses integrity and security scanning of software artefacts before deployment. These controls cover the supply chain from build to deployment.

Gap: Agent-generated first-party code is a novel supply chain input — it's code that appears in-house but was produced by an external system (the AI model). The artefact integrity controls don't have a category for "first-party code generated by a third-party system." The risk properties are also different: third-party components have independent defect distributions, while agent-generated code has correlated defects (Section 2.4). The controls verify artefact integrity but not the semantic correctness of the code within those artefacts.

6.1.2 Controls with No Coverage

The following gap areas have no corresponding ISM control. Each represents a category of agentic risk that falls outside the current framework's scope.

Agent output as trust boundary — *ACF-T1 (trust tier conflation), ACF-E1 (implicit privilege grant)*

No control addresses the trust classification of AI-generated artefacts. Agent code is neither "in-house" (human-authored) nor "third-party" (external component) — it's a new category. ISM-2074 (Dec-25) requires organisations to develop, implement, and maintain an AI usage policy, but this is a governance control, not a technical trust boundary control.

Review capacity scaling — *ACF-D1 (finding flood), ACF-D2 (review capacity exhaustion)*

ISM-2060 and ISM-2061 mandate code review and security-focused peer review, but neither addresses what happens when code generation velocity exceeds review capacity. No control

requires organisations to demonstrate that review remains effective under volume pressure.

Semantic boundary enforcement — *ACF-S1 (competence spoofing), ACF-S3 (structural identity spoofing), ACF-T1 (trust tier conflation), ACF-T2 (silent coercion)*

No control addresses the gap between syntactic correctness and semantic correctness in the context of trust boundaries. ISM-0402's SAST/DAST/SCA requirement covers known vulnerability patterns but not context-dependent semantic correctness. Existing controls assume that if code passes review and testing, it is adequate.

Correlated failure detection — *All ACF categories*

No control addresses the distinct risk profile of correlated defects. Testing and review strategies are designed for independent failure distributions.

Code provenance tracking — *ACF-D2 (review capacity exhaustion)*

No control requires organisations to track which code was generated by AI agents vs. authored by humans. ISM-2074 requires an AI usage policy but not per-artefact provenance. Without provenance, risk assessment cannot distinguish between code populations with different failure characteristics.

6.1.3 Candidate ISM Extensions

The following are illustrative extensions, not formal proposals. They are included to demonstrate that the gaps are addressable within the ISM's existing structure. The use of SHOULD below follows the ISM's existing convention for conditional controls — it is illustrative wording to show how these extensions might read, not normative language.

Extension to ISM-0401 (Secure by Design):

> When AI agents are used to generate code for assessed systems, the organisation's Secure by Design practices SHOULD include machine-enforceable rules for trust boundary maintenance, defensive pattern restrictions appropriate to the system's data sensitivity, and audit trail preservation requirements. Secure by Design principles that exist only as human-readable documentation are insufficient controls against AI-generated code, which does not read documentation.

Extension to ISM-2060/2061 (Code Review and Security-Focused Peer Review):

> When AI agents generate a significant proportion of code changes, the organisation SHOULD demonstrate that its code review process (ISM-2060) and security-focused peer review process (ISM-2061) remain effective at detecting semantic defects — not merely syntactic or conventional defects — under the volume of changes produced. Evidence may include measured defect escape rates, review depth audits, or demonstrated use of automated semantic pre-screening that reduces the burden on human reviewers.

New control (Agent Output Trust Boundary):

> Code generated by AI agents SHOULD be treated as external input requiring validation at the boundary before integration into assessed systems. The organisation SHOULD define and document the validation boundary, including what properties are verified (trust boundary maintenance, audit trail integrity, error handling appropriateness) and what evidence demonstrates the validation is effective.

New control (Code Provenance):

> When AI agents are used in the development of assessed systems, the organisation SHOULD maintain records of which code was generated by AI agents, which was human-authored, and which was agent-generated then human-modified. This provenance metadata supports risk assessment, incident response, and targeted remediation when systematic agent-introduced defects are discovered.

6.2 NIST Secure Software Development Framework (SSDF)

SP 800-218 defines practices for secure software development organised into four groups. The analysis below maps each group to agentic code concerns:

Practice Group	SSDF Practices	Agentic Code Coverage
Prepare the Organization (PO)	Define security requirements, roles, training	Does not address training requirements for reviewing agent output (which requires different skills than reviewing human output) or organisational capacity planning for agent-scale review volume
Protect the Software (PS)	Protect code, integrity verification	Addresses integrity of code artefacts but not the trust classification of code based on its generation method. An agent-generated commit and a human-authored commit are indistinguishable in the VCS
Produce Well-Secured Software (PW)	Design, code review, testing	The most relevant group. Practice PW.5 (secure coding practices), Practice PW.7 (code review), and Practice PW.8 (testing) all partially apply. However, PW assumes a human developer who can be trained, who learns from feedback, and whose error rate is independent across functions — none of which hold for agents
Respond to Vulnerabilities (RV)	Vulnerability response, disclosure	Does not address the correlated nature of agent-introduced defects. Standard vulnerability response treats each finding independently; agent defects require pattern-wide remediation (Section 9.7)

Key SSDF gap: Practice PW.1 ("Design Software to Meet Security Requirements and Mitigate Security Risks") includes task PW.1.1, which recommends "using forms of risk-based analysis to determine how much effort is adequate" for security practices. This implicitly assumes that risk is assessable per-component. Agent-generated code introduces *systematic* risk across many

components from a single source — the analysis framework needs to account for correlation, not just per-component risk.

NIST's own recognition of this gap: NIST published SP 800-218A (July 2024) as a supplement to the SSDF specifically for generative AI and dual-use foundation model development contexts, acknowledging that the original framework's human-centred assumptions need AI-specific augmentation. However, SP 800-218A's focus is on secure practices for AI *model* development across the SDLC — not on the assurance of source code *generated by* AI systems. This is precisely the gap this paper addresses: substantial guidance now exists for building AI safely, but almost none for securing what AI builds.

6.3 Essential Eight

The Essential Eight maturity model does not directly address software development practices. However, two strategies are relevant by analogy:

Application Control establishes that not all software should be trusted equally based on its source. The maturity levels (ML1: prevent execution of unapproved programs; ML2: restrict to approved directories; ML3: comprehensive control) provide a model for graduated trust that could be extended to code generation sources. Analogy: agent-generated code is "unapproved software" until it passes through a validation boundary — similar to how an unsigned binary is untrusted until it meets the application control policy.

Restrict Administrative Privileges establishes the principle of least privilege. Applied to agentic coding: agents should not have the ability to modify security-critical configuration (e.g., allowlists, audit configuration, access control rules) without human approval. This maps directly to the `CODEOWNERS` protection and temporal separation mechanisms described in Appendix B.

6.4 OWASP and Industry Guidance

OWASP Top 10 for LLM Applications (2025) primarily addresses threats *to* LLM systems — prompt injection, training data poisoning, model denial of service. The closest entry to this paper's concerns is **LLM02 (Insecure Output Handling)**, whose attack scenarios explicitly include LLM-generated code introducing vulnerabilities such as SQL injection. However, even LLM02 frames this as an application-level output-handling problem — advising developers to treat LLM output with "zero-trust" validation — rather than providing a comprehensive treatment of the distinct failure characteristics of AI-assisted code generation (correlated defects, review capacity exhaustion, context-inappropriate patterns). The project has since evolved into the broader **OWASP GenAI Security Project**, covering LLM applications, agentic AI systems, and AI-driven applications — but no OWASP project specifically targets the assurance of AI-generated source code in government systems in the sense addressed by this paper.

OWASP Secure Coding Practices provides a checklist of defensive coding practices. Ironically, several "secure" practices in the OWASP checklist are precisely the anti-patterns that the agentic threat model identifies as dangerous in high-assurance contexts. For example, "validate all input" is correct, but "provide a default value when input is missing" is context-dependent — in audit-critical systems, a missing value should crash, not default. This illustrates the gap between generic secure coding guidance and domain-specific trust boundary requirements.

MITRE ATT&CK and CWE provide taxonomies for attack techniques and code weaknesses respectively. The agentic code failure modes in Appendix A do not map cleanly to existing CWE entries because they are not individual weaknesses — they are *patterns* that are correct in most contexts and dangerous in specific ones. A `.get()` with a default value is not a weakness; it is

a weakness *when applied to audit-critical data in a system that requires crash-on-corruption*. Context-dependent weaknesses are not well-served by context-free taxonomies.

6.5 The Gap Between "Securing AI" and "Securing What AI Builds"

Across all current frameworks, there is a consistent structural gap: substantial guidance exists for securing AI systems themselves (the model, the training pipeline, the inference infrastructure), but almost no guidance exists for securing systems that AI systems build or modify.

This gap is not surprising — agentic coding is a recent capability and guidance takes time to develop. But the gap is widening faster than it is closing, because:

- Agent adoption is accelerating (driven by generation velocity gains and expanded capability — Section 1.2, Section 8)
- The failure modes are subtle (they pass all existing automated checks — Section 2.2)
- The vocabulary for discussing these failures doesn't exist yet in policy contexts (this paper's taxonomy is a first attempt)

6.6 Detection Coverage Is Worst Where Risk Is Highest

Before listing the structural gaps, it is worth stating the detection picture plainly. The ACF taxonomy (Appendix A) catalogues thirteen agentic code failure modes. Of those thirteen:

Detection Level	Count	Implication
None — no existing tool detects it	5	Requires new tooling or new review practices
Partial — some tools catch some cases	5	Existing tools provide incomplete coverage
Good — existing tools generally catch it	1	Already addressed by current tooling
N/A — process threat, not code pattern	2	Requires process controls, not technical controls

Five of thirteen failure modes have zero tool coverage. Both Critical-rated entries — ACF-T1 (trust tier conflation) and ACF-E1 (implicit privilege escalation) — are among the five with no detection capability whatsoever. The highest-risk failures are precisely the ones current tooling misses entirely. The full detection breakdown with specific failure IDs is in Appendix A.

This is the gap in quantitative terms. The structural gaps follow.

6.7 Structural Gaps

No current framework provides:

1. **A taxonomy of agentic code failure modes** grounded in established threat modelling (STRIDE or equivalent)
2. **Controls for semantic correctness** beyond syntactic and functional correctness — trust boundary maintenance, audit trail integrity, context-appropriate error handling
3. **Controls for review effectiveness at scale** — not just "is code reviewed?" but "does the review process remain effective at agent-generated volume?"

4. **Trust classification for agent output** — how should agent-generated code be treated in the system's trust model?
5. **Accreditation criteria for agentic development workflows** — what evidence must organisations provide to demonstrate that agentic coding maintains the required security posture?
6. **Vocabulary for context-dependent code weaknesses** — patterns that are correct in general but dangerous in specific security contexts
7. **Correlated failure risk models** — testing and remediation strategies that account for the non-independent failure distribution of agent-generated code
8. **Governance perimeter expansion** — controls for executable logic produced by non-developers using agentic tools outside traditional SDLC channels (Section 1.2.8)

Current frameworks scope software development controls to recognised development teams and established code repositories. Agent-generated automations, integrations, and plugins produced by analysts and operators outside these channels are not addressed by any current guidance.

7. The Response Landscape

The responses available to organisations fall into three categories of increasing assurance strength (ordered weakest to strongest):

Control Type	Mechanism	Strength	Example
Behavioural	Relies on individual compliance	Weakest — requires sustained restraint against incentives	"Developers should not run more than one agent concurrently"
Procedural	Relies on organisational process	Moderate — requires consistent enforcement and audit	"Parallel agent-generated changes require separate review queues and staged approval"
Technical	Constrains the environment	Strongest — operates regardless of individual behaviour	"The CI/CD pipeline enforces concurrency limits, sequencing rules, or protected-branch gates for agent-originated changes"

Most organisations will implement behavioural controls, aspire to procedural controls, and underinvest in technical controls — because technical controls constrain the velocity that motivated adoption. The key insight from security engineering applies here: **controls that shape the environment are stronger than controls that depend on restraint.** A rule that developers must not bypass review is an aspiration; a pipeline that physically prevents unreviewed code from reaching protected branches is a control.

The sections below are ordered from weakest to strongest assurance, not from least to most important. All three have a role, but assurance should not rest primarily on behavioural or procedural controls where technical enforcement is feasible. Organisations that rely on behavioural and procedural controls without technical enforcement should understand that their assurance argument rests on sustained human compliance with rules that run directly against the productivity incentive that makes agentic development attractive.

7.1 Process Controls (Strengthening Existing Practices)

Current best practice, adapted for agentic velocity:

Enhanced code review protocols

Mandate security-focused review (not just correctness review) for agent-generated code. Require reviewers to attest that trust boundaries were verified, not just that the code "looks right." This is a process change, not a technology change, but it requires explicit recognition that agent code needs different review criteria than human code.

Specifically, review checklists for agent-generated code should include:

- Were trust boundaries maintained? (Does external data pass through validation before internal use?)
- Are error handlers audit-preserving? (Do `except` blocks propagate to the audit system, or swallow and continue?)
- Are default values justified? (Does every `.get()` or `getattr` with a default represent a legitimate design decision, or a fabrication of missing data?)
- Is the code's failure mode correct for the context? (Should this code crash, quarantine, or continue on error?)

Separation of generation and review

The person (or agent) who generates the code must not be the sole reviewer. This already applies to human-authored code in most government contexts; extending it to agent-generated code means ensuring that agent self-review (e.g., an agent checking its own output) does not count as an independent review. This has a subtlety: multi-agent workflows where one agent generates code and another reviews it using different models are not independent review in the statistical sense — the models share overlapping training corpora and failure modes. However, a more nuanced form of multi-agent review offers meaningful value.

Structured perspective diversity in agent-assisted review

While model diversity (using different models) provides limited independence, *perspective diversity* — prompting the same model with different analytical frames — can produce meaningfully different coverage. One useful way to organise these perspectives is by *cognitive function* rather than by domain. Illustrative functions include: an **architectural reviewer** ("Is the shape right?"), a **problem-framing reviewer** ("Is this solving the right problem at the right location?"), an **implementation reviewer** ("Was it implemented correctly?"), and a **quality reviewer** ("Are the tests and verification strategy adequate?"). These lenses are not independent, but they surface different classes of issue. A trust boundary violation (ACF-T1) may be an implementation defect; the *reason* it exists may be an architectural misplacement — a validation that belongs in a different layer; and the tests may still pass while verifying the wrong thing. The problem-framing perspective is particularly underexplored: evaluating whether a change addresses the root cause or patches a symptom often requires holding broader dependency context in working memory — something agents may assist with and time-pressured human review often struggles to sustain.

This is not independence — the underlying model's blind spots persist across all frames — but it is *faceted analysis* that surfaces different failure classes. In practice, a small set of prompted perspectives may provide broader first-pass coverage at agent speed than a single undifferentiated review pass, reducing the volume of issues that reach human review while increasing the diversity of issues caught. Organisations should develop *role-specific review prompts* aligned to their threat model and architecture — not generic "review this code" instructions. The prompts themselves become reviewable, version-controlled security artefacts: institutional knowledge encoded in a different form than the machine-readable rules described in Section 7.2, but serving a complementary function.

The honest caveat: this is still a procedural control, not a technical one. It depends on someone actually running the prompted reviews, and the quality depends on prompt design. It sits in the middle tier of the control strength hierarchy in this section. But it is achievable now with no tooling investment, which makes it a useful bridge while automated enforcement tooling matures.

Volume-aware review capacity planning

If agents materially increase code generation volume, review capacity must be addressed — either through additional reviewers, automated pre-screening that reduces the human review burden, or rate-limiting agent output to match review capacity. Ignoring the volume mismatch means the review control degrades silently.

Project-level instructions as a generation-time control

Most agent frameworks support project-level configuration — system prompts, instruction files, memory stores — that shape agent behaviour within a session. These instructions can encode project-specific rules ("never use `.get()` on audit data," "all error handlers for audit-write operations must propagate, not catch-and-continue") and reduce the frequency of the failure modes in Appendix A at generation time. This is a behavioural control (weakest tier): it depends on the agent respecting the instructions, the instructions being comprehensive enough to cover all failure modes, and the agent not generalising incorrectly from specific rules. Instructions that say "don't use `.get()` on audit data" do not teach the agent the underlying principle "don't fabricate defaults where data absence is meaningful" — every specific rule must be spelled out. Nonetheless, project-level instructions are an immediately deployable, zero-cost control that measurably reduces (without eliminating) the volume of semantic violations reaching the review pipeline.

Provenance tracking for agent output

Organisations should maintain records of which code was generated by agents, which was human-authored, and which was agent-generated then human-modified. This metadata is relevant for both security assessment (understanding the trust profile of different code regions) and for incident response (when a defect is found, knowing whether it originated from agent generation helps diagnose the failure mode).

7.2 Technical Controls (What's Buildable)

When organisations evaluate the feasibility of security tooling for agentic code, the instinctive question is: *"How big is it? How many lines of code? How many hours to build?"* This is the wrong question. It inherits the assumption that scope correlates with assurance — that a 200-line tool provides less assurance than a 2000-line tool, or that 220 hours of development is insufficient for a security-critical control.

The right question is: **"How do you know it's correct?"**

This question applies recursively. If the security enforcement tool is itself built by an agent (which is increasingly likely for any new tool), then the tool's correctness is subject to the same threat model it exists to address. The verification mechanism — not the implementation size — is the assurance argument.

This is not a new problem. Test suites that consolidate object construction into shared factories face the same trade-off: centralising institutional knowledge (what a valid test object looks like) eliminates a class of bugs caused by ad hoc misconfiguration, but concentrates risk in the factories themselves. A defect in a factory propagates to every test that depends on it. The engineering response is not to avoid centralisation — the alternative is worse — but to verify the central artefacts with disproportionate rigour. The same principle applies to any tool that

encodes institutional security knowledge: the value of centralisation is high, but so is the cost of getting the central artefact wrong.

This reframing has direct implications for policy:

- **Accreditation should evaluate the verification story**, not the implementation scope. A small tool with a rigorous golden corpus, self-hosting gate, and measured precision is stronger than a large tool without these properties.
- **"How do you know the agent wrote it correctly?"** is the question that applies to all agent-generated code, including the tools that check agent-generated code. The answer must be grounded in independent verification (test corpus, self-hosting, measured false positive/negative rates), not in the development process used to produce it.
- **The line between "tool" and "assurance argument" blurs.** The design of a semantic boundary enforcer (Appendix B) is essentially a structured answer to "how do you know the code is correct?" — and that answer is valuable independent of whether the specific tool gets built.

This reframing directly informs Recommendations 7 and 8 — accreditation should evaluate the verification story, and IRAP assessors should assess review quality, not just review process. The technical controls below should be evaluated on this basis.

Automated semantic boundary enforcement. Static analysis tools that verify data provenance and trust tier flow — not just type shape — at the code level. These tools check that external data passes through validation boundaries before reaching internal processing, that error handling preserves audit trails, and that defensive patterns are not used on data that should crash on anomaly.

Technical feasibility finding: The case study codebase (Section 8) runs a pattern-matching enforcement gate that catches trust boundary violations in CI. Building on this production experience, an enhanced semantic boundary enforcer for Python has been designed (see Appendix B) with the following properties:

- Zero external dependencies (uses only Python's standard library AST module)
- Works with standard Python — no language modifications, no runtime dependencies
- Produces findings in SARIF format (industry standard for static analysis results)
- Can operate as a pre-commit check, CI gate, or agent self-check before code submission
- Verification properties (golden corpus, self-hosting gate, measured precision) are independently auditable — see Appendix B, Section B.5

This is not the only possible technical control, but it demonstrates that the problem is tractable. Existing static analysis platforms — Semgrep (custom rule authoring), CodeQL (dataflow analysis), and Pysa/Pyre (taint tracking) — could be extended with rules targeting the ACF taxonomy's failure modes, particularly trust tier flow (ACF-T1) and validation boundary gaps (ACF-T2). The design in Appendix B chose AST-level analysis with zero dependencies for deployment simplicity, but organisations with existing static analysis infrastructure should evaluate whether their current tools can be adapted before building new ones.

Agent-assisted semantic analysis. The prompted perspective diversity described in Section 7.1 applies not only to reviewing changes but to *analysing existing code*. An agent prompted with a specific analytical frame can perform a cold read of a source file — tracing execution flow, following data through trust boundaries, and evaluating error-handling paths — with a level of context-sensitive analysis that traditional static analysis often cannot match and that human reviewers cannot sustain at scale. Static analysers are strongest at structural, syntactic, and

formally encoded dataflow patterns; they cannot reliably determine whether a `.get()` default is *contextually appropriate* without project-specific semantic rules. Human reviewers can make that judgement, but cannot practically sustain file-by-file semantic tracing across multiple analytical lenses at codebase scale. Agent-assisted analysis therefore occupies a previously sparse middle ground: semantic inspection applied at a scale that was economically impractical for human review alone. This is not a replacement for either static analysis or human review — it catches a different class of issue from both, and its blind spots differ from both. In most settings, it is likely to be most useful as a non-blocking discovery control, complementing CI-integrated static analysis and targeted human review rather than replacing either.

In the case study deployment (Section 8), periodic full-codebase agent crawls — every file analysed through prompted analytical frames — routinely surfaced dozens of findings per pass, the majority of which would not have been caught by conventional static analysis, incremental code review, or unprompted agent review. The findings emerge specifically *because* the analytical frames are non-default — neither human reviewers under time pressure nor agents given generic review instructions naturally adopt the perspective of, say, an architectural reviewer evaluating layer responsibility or a problem-framing reviewer asking whether a fix addresses root cause or symptom. This points to a human limitation that is distinct from time pressure: *cognitive range*. A senior backend engineer cannot practically adopt a PyTorch engineer's analytical frame, or a security architect's, or a data quality auditor's — not because they lack intelligence, but because genuine expertise in each frame takes years to develop. Prompted agents do not have deeper expertise in any single frame than a domain specialist, but they can adopt multiple frames without the switching cost and domain-knowledge barriers that make it impractical for any individual human reviewer to cover more than one or two perspectives well. In this sense, prompted agents are *polymorphic* reviewers — not deeper than a specialist in any single frame, but able to provide breadth of analytical coverage that no individual human can match regardless of time budget. Most findings are individually low-impact — edge cases in cold paths, minor deviations from stated invariants, inconsistencies that do not currently trigger in testing — but they represent legitimate deviations from the codebase's architectural rules, and in an auditable system, each deviation is a gap in the assurance argument regardless of its current exploitability. The cost is non-trivial (such crawls consume substantial API quota), but the defect yield suggests the economics may favour periodic comprehensive analysis over exclusive reliance on per-change review.

Key architectural principle — "parasitic, not parallel": Effective tools for this space must extend existing programming language machinery (annotations, type hints, decorators) rather than creating parallel systems that require adoption of new syntax or tools. Tools that require developers to learn a new language or adopt a new framework face adoption resistance that undermines their security value. Critically, enforcement must live inside the existing CI/CD pipeline — pre-commit hooks, CI gates, pull request checks — not in a separate workflow. If a security tool slows down the very velocity the organisation bought the AI agent to achieve, the tool will be bypassed. The enforcement mechanism succeeds by being invisible to the fast path and blocking only on genuine violations.

7.3 Policy Controls (What Doesn't Exist Yet)

Standardised vocabulary. The taxonomy in this paper (Section 3 and Appendix A) provides a starting point. Government cybersecurity guidance needs terminology for agentic code failure modes — "competence spoofing," "trust tier conflation," "audit trail destruction through defensive patterns" — that practitioners can use in security assessments, risk registers, and accreditation documentation.

Accreditation criteria for agentic development workflows. IRAP assessments and similar accreditation processes need criteria for evaluating whether an organisation's use of AI coding agents maintains the security posture required by the system's classification. This includes:

- How agent output is validated before integration
- How review effectiveness is maintained under volume pressure
- How trust boundaries are verified in agent-generated code
- What attestation is required from human reviewers

Agent output classification. A formal determination of how agent-generated code should be treated in the trust model — as external input (Tier 3), as tool output (requiring specific validation), or as a new category requiring its own controls.

Extend SDLC-equivalent controls to executable logic produced outside formal development teams. Organisations should inventory and govern agent-generated plugins, automations, BI extensions, workflow scripts, low-code components, and similar artefacts produced by analysts, operators, and other non-developer staff (see Section 1.2.8). Where such artefacts affect trust boundaries, access control, audit trails, or data integrity, they should be subject to provenance, review, and validation controls proportionate to their impact — even when the producers do not consider themselves developers and the artefacts do not live in a formal version control system. This is the policy response to the governance perimeter problem: the SDLC boundary has expanded, and controls must follow it.

8. Case Study: Agentic Development Under Compliance Constraints

This section presents a composite, de-identified account drawn from experience with agentic development in compliance-constrained environments. Specific implementation details have been generalised. The system and tooling described are open-source, Commonwealth-owned assets; they are de-identified here to keep focus on the generalisable threat model rather than implementation-specific design choices.

8.1 Context

An engineering team developing an auditable data processing platform — a system where every decision must be traceable to its source data, configuration, and code version. The system processes sensitive data under compliance requirements that mandate complete audit trails, data integrity verification, and defence-in-depth security controls.

The team adopted AI coding agents as a primary development tool, with agents generating the majority of new code. The codebase enforces strict architectural rules: a tiered trust model for data handling, mandatory crash-on-corruption for internal data, quarantine-and-continue for external data, and no defensive programming patterns (no `.get()` on typed objects, no bare `except`, no silent error swallowing).

These rules are documented extensively but are **institutional knowledge** — they exist in project documentation, not in the programming language. Python permits all of the patterns the project forbids.

8.2 The Compliance Tax

Compliance requirements impose substantial overhead on the development workflow:

Hash integrity verification. Every data operation produces cryptographic hashes for audit trail integrity. Changes to hashing logic, serialisation formats, or data structures require recompilation and verification of hash chains. This is mechanical work that agents handle well, but the *verification* that the agent produced correct hashes requires human review — or automated checking.

Audit trail completeness. Every code path must produce audit records. Missing audit records are treated as evidence tampering, not as bugs. This means error handlers cannot swallow exceptions, partial operations must roll back, and "log and continue" is not an acceptable failure mode. Agents consistently produce error handlers that violate these requirements because their training data overwhelmingly models "log and continue" as best practice (Section 2.5).

Trust boundary enforcement. The team maintains a static analysis tool (manually built, project-specific) that detects defensive coding patterns in contexts where crash-on-corruption is required. This tool has an allowlist mechanism for legitimate exceptions. Maintaining the allowlist, reviewing new findings, and ensuring agents don't introduce violations that the tool doesn't yet detect is a recurring cost.

The overhead is real — the team estimates the aggregate compliance overhead at 15-25% of total development time — but its distribution is uneven and reveals something about how agents experience unfamiliar workflows. On large changes (new features, multi-file refactors), compliance overhead is trivially small relative to the work — even if the agent spends 10 minutes recalculating 500 allowlist hashes after a major refactor, that is a reasonable investment against hours of productive generation. But on small changes, the ratio inverts dramatically. An agent

fixing a one-line bug spends 30 seconds on the fix and 60 seconds grappling with the CI pipeline: discovering that an enforcement gate is blocking the commit, parsing the error to understand what the tool expects, calculating a new allowlist hash, and editing the configuration. The compliance work takes twice as long as the actual work. The agent still completes the entire cycle faster than a human would, but the unfamiliar workflow — one the agent has never seen in training data and must rediscover each session — imposes an outsized cost on small tasks, and this skew toward small-change cases is where the bulk of the overhead concentrates.

This is not new overhead introduced by agentic coding. It is the *same compliance overhead* that existed before agents, redistributed. Before agents, humans spent that time writing compliant code slowly. With agents, humans spend it reviewing agent code for compliance quickly. The total compliance cost is similar; the development velocity is dramatically higher.

8.3 Productivity Gains Despite Friction

Despite the compliance overhead, the team reports substantial productivity gains from agentic coding:

- **Mechanical refactoring** (renaming, restructuring, pattern application across files) is handled almost entirely by agents. This work is tedious for humans but trivial for agents, and the compliance requirements don't make it harder — the agent produces the refactoring, the existing test suite and CI pipeline verify it.
- **Boilerplate generation** (new plugins, test scaffolding, configuration structures) is dramatically accelerated. The agent follows the project's existing patterns, and the structural correctness is verifiable by tests.
- **Bug investigation and test writing** benefit from agents' ability to rapidly explore code paths and generate test cases. The agent's lack of institutional knowledge is less of a liability when the task is "write a test that exercises this code path" rather than "write code that maintains trust boundaries."

The pattern is clear: **agents excel at tasks where correctness is structurally verifiable** (tests pass, types check, linter is clean) and struggle at tasks where **correctness requires institutional knowledge** (trust boundary maintenance, audit trail completeness, appropriate error handling in compliance contexts).

8.4 Where the Current Process Fails

The team has observed agent-generated code of questionable quality passing human review. In each case, the code was:

- Syntactically correct
- Consistent with project conventions
- Passing all existing tests
- Semantically wrong in the compliance context

The failure modes map directly to the taxonomy in Section 3:

Competence spoofing (ACF-S1). Agent generates `.get()` with a default value on a data structure where a missing field indicates a critical failure in an upstream internal component — absence is evidence of corruption, not a case to handle gracefully. The code appears defensive and robust. A reviewer under time pressure sees "handles the missing case" rather than "fabricates data where absence means something has gone seriously wrong upstream and continuing with a default is irresponsible."

Audit trail destruction (ACF-R1). Agent wraps an audit-critical operation in a `try/except` that logs the error and continues. The code appears to handle errors gracefully. The reviewer doesn't recognise that the caught exception should propagate to the audit system rather than being logged and swallowed.

Trust tier conflation (ACF-T1). Agent deserialises data from an external API and passes it directly to an internal processing function. The code appears clean — no obvious security issues. The reviewer doesn't see the missing validation boundary because both the external data and internal data are the same Python type (`dict`).

In each case, the defect was caught later — by the project's existing static analysis tool, by a different reviewer examining adjacent code, or by a test failure in a downstream component that received malformed data. The initial review process, which was supposed to catch these issues, had signed off.

The project's automated semantic boundary enforcement tool (Section 7.2; see Appendix B for the proposed enhanced design) provides a rough empirical signal. In steady-state development on a ~80,000-line Python codebase with agents generating the majority of new code, the CI gate catches approximately 1–2 trust boundary violations per day — patterns from the ACF taxonomy (primarily ACF-S1, ACF-R1, and ACF-T1) that the generating agent introduced. These violations never enter the repository. They are caught at the pre-commit or CI boundary before a human reviewer ever sees them. Without the gate, they would enter the codebase through normal code review, because they look like correct defensive Python — which is exactly what makes them dangerous (Section 2.3). Over a quarter, this accumulates to roughly 90–180 violations that automated tooling caught at the boundary.

Three aspects of this number merit attention. First, it represents only the violations the tool detects — the tool's coverage of the ACF taxonomy is incomplete (Appendix A, Detection Capability Summary), so the true rate of semantic violations is higher. Second, and more important for the policy argument: these are not exotic edge cases. They are the routine, daily output of a well-prompted agent operating on a well-documented codebase with project-level instructions that explicitly prohibit the patterns in question. The agent still produces them, because the patterns are deeply embedded in its training data — and because the agent has no persistent memory across sessions (Section 2.4), the same patterns recur regardless of how many times they have been caught previously. Third, the enforcement model is not advisory — it is a gate with an allowlist. A pattern flagged by the enforcer either gets fixed by the agent or requires a human-authored exception with a rationale, a reviewer identity, and an expiry date (Appendix B, Section B.4). Legitimate uses of otherwise-restricted patterns go through; unconscious pattern completion from training data does not.

The question for organisations without this kind of enforcement is not whether these patterns exist in their agent-generated code — it is whether anything is catching them.

8.5 The Redirection Insight

The team's experience reveals that automated semantic enforcement doesn't *add* tedium — it **redirects existing tedium** toward higher-value activities.

Without automated enforcement, humans manually review every agent output for trust boundary violations. This is:

- **Error-prone:** The failure modes look like correct code (Section 3)
- **Fatigue-inducing:** Reviewing dozens of agent-generated functions per day for subtle semantic violations degrades review quality (Section 4.2)
- **Unscalable:** As agent velocity increases, review capacity doesn't

With automated enforcement, the machine catches structural trust boundary violations (unterminated taint paths, defensive patterns on typed objects, missing validation boundaries). Humans focus on **semantic issues that require institutional knowledge** — whether the trust topology is correctly declared, whether the validation logic is actually correct (not just structurally present), whether the audit trail captures the right information.

This is a genuine improvement in security posture, not just efficiency:

Review Focus	Without Automation	With Automation
"Is `.get()` used on typed objects?"	Human scans for pattern (error-prone)	Machine catches structurally (reliable)
"Does this error handler preserve the audit trail?"	Human evaluates (moderate difficulty)	Machine flags broad `except` blocks; human evaluates the specific cases
"Is the trust topology correctly declared for this new module?"	Human evaluates (requires institutional knowledge)	Human evaluates (no change — this is irreducibly human)
"Is this validation function actually validating?"	Human evaluates (requires domain knowledge)	Machine checks structural presence of control flow; human evaluates semantic adequacy

The total review burden may be similar, but the **distribution of human attention shifts** from low-value pattern scanning to high-value semantic evaluation. The compliance tax is the same; the assurance yield is higher.

A less obvious but equally important effect: **more compliance work is actually executed in an agentic workflow than in a purely human one.** The same property that makes agents dangerous — no persistent learning, no internalised shortcuts — makes them unusually compliant enforcement subjects. A developer fixing a one-line bug at 4:55pm on a Friday is not going to recalculate 500 allowlist hashes, rerun the full CI pipeline, and write a whitelist rationale for the `.get()` they just introduced. They will commit, push, and deal with it Monday. The agent doesn't have a Friday. It doesn't have a deadline. It lacks the judgment to decide "this governance step isn't worth it for a one-liner." It just does the work. Every session it hits the pre-commit hook fresh, grapples with it fresh, and either satisfies it or fails. It never develops the human instinct of "I'll fix that later" — because there is no later for it.

This is the mirror image of the persistent-learning problem described in Section 2.4. The agent doesn't learn your security rules, but it also doesn't learn which rules it can get away with skipping. The net effect is that the agent pays the governance tax that humans under deadline pressure quietly defer. For anyone who has audited a development team and found the gap between "documented process" and "what actually happens under delivery pressure," this is a significant finding: agents are simultaneously the worst authors and the best compliance subjects.

This also adds nuance to the control-strength hierarchy in Section 7. The paper argues behavioural controls are weakest because they require sustained restraint against incentives. That is true for humans. For agents, behavioural controls are weak for a *different* reason — not because the agent will choose to skip them, but because it won't remember them next session. Technical controls (CI gates, pre-commit hooks) are strong for agents for the same reason they are strong for humans — they are environmental, not volitional — but they gain an additional benefit: the agent won't resent the gate, won't lobby to have it removed, won't find a workaround, and won't develop learned helplessness about false positives. It will run into the wall, fix the issue, and move on. Every single time.

The underlying lesson from the case study is this: **agentic development is viable precisely because the agent will execute governance that humans under pressure quietly defer — but it requires governance designed for the agent's actual failure modes, not the human's.** Human governance assumes the author remembers last session's feedback, can be trained over time, and will exercise judgment about which rules matter. Agent governance must assume none of those things. It must be environmental (CI gates, not documentation), boundary-enforced (pre-commit, not post-review), and stateless (every session is the first session). Organisations that apply human-shaped governance to agents will get the agent's compliance without catching the agent's mistakes. Organisations that design agent-shaped governance will get both.

9. Open Questions for the Community

This paper does not attempt to answer these questions definitively. They are posed for discussion and community input.

9.1 Trust Classification

Should agent-generated code be treated as a distinct trust tier in security assessments?

The Tier 3 (external/untrusted) classification proposed in Section 5 is a starting point, but it may be too coarse. Agent code that has been reviewed and modified by a human developer exists in an intermediate state — it originated externally but has been validated. Does this validation elevate it to Tier 2? What constitutes sufficient validation?

A related question: does the trust classification of agent code change over time? Code generated by an agent 12 months ago that has been in production, passed multiple review cycles, and survived real-world use arguably has a different trust profile than code generated 5 minutes ago. Should the trust tier be static (based on origin) or dynamic (based on accumulated evidence)?

9.2 Accreditation Burden

What is the appropriate accreditation burden for organisations using agentic coding in PROTECTED or higher systems?

Current IRAP assessment criteria do not specifically address AI code generation. Should they? If so, what evidence should an organisation provide to demonstrate that its use of agents maintains the required security posture?

Candidate evidence requirements (for discussion):

- Demonstration that agent output passes through a defined validation boundary (Section 5.3)
- Measured review effectiveness metrics (not just “we have a review process” but “here is evidence the process catches the failure modes in Appendix A”)
- Provenance records showing which code was agent-generated (Section 7.1)
- Evidence that automated semantic enforcement tools are in use and their precision is measured (Appendix B, Section B.5)

Must the validation boundary itself be assessed? If an organisation builds or adopts automated enforcement tooling, does that tooling become part of the assessed system?

9.3 Static vs. Runtime Assurance

Can static analysis tools provide sufficient assurance for agent-generated code, or is runtime verification necessary?

Static analysis can detect structural patterns (Section 7.2) but cannot verify semantic correctness in all cases. Runtime verification (monitoring trust tier flow at execution time) provides stronger guarantees but introduces performance overhead and operational complexity. Where is the appropriate trade-off for government systems?

The design work in Appendix B suggests a middle ground: static analysis as the primary enforcement mechanism with narrow runtime verification in test mode only — calibrating the static analyzer’s coverage without introducing production runtime dependencies. Whether this trade-off is acceptable for a given classification level is a policy question, not a technical one.

9.4 Review Quality Measurement

How do you measure the effectiveness of human review when the volume of agent output makes comprehensive review impractical?

Current accreditation processes assume that code review provides a certain level of assurance. If review quality degrades under volume pressure (Section 4), the assurance claim is no longer valid. How should organisations measure and demonstrate that their review process remains effective?

Possible metrics (all have limitations):

- **Defect escape rate:** How many agent-introduced defects are found post-review? Requires knowing the total defect count, which requires a detection mechanism independent of the review process.
- **Review depth sampling:** Periodically audit review decisions for thoroughness. Resource-intensive but directly measures quality.
- **Automated pre-screening coverage:** What percentage of the failure modes in Appendix A are caught by automated tools before reaching human review? This doesn't measure review quality directly but measures how much the review process is being supplemented.
- **Time-per-change metrics:** Review duration per lines changed. A leading indicator — if review time per change is declining while change volume increases, review quality is likely degrading.

9.5 Agent Self-Regulation

Should agents be required to self-check their output against security rules before submitting it?

Many agent workflows already include self-checking loops — agents run linters, type checkers, and test suites against their own output before submission, often triggered by pre-commit hooks or CI integration. This is current practice, not a future possibility. The question is whether this practice should be *required* and whether it should extend to semantic security checks.

Pre-generation self-checking (where the agent validates its own output against security rules before submission) closes the feedback loop at the point of maximum leverage — before the code enters the repository. This is technically feasible (Appendix B describes a `--stdin` mode for exactly this purpose).

But current practice and the proposal both raise questions:

- Does self-checking constitute validation, or must validation be independent? The agent checking its own output against structural rules is different from the agent evaluating whether its output is semantically correct — the first is a mechanical verification, the second is the same judgment that produced the code.
- Can an agent meaningfully check for the failure modes it is predisposed to produce? If the agent's training data biases it toward `.get()` with defaults, will it also generate code that satisfies a rule against that pattern, or will it generate code that evades the rule while preserving the same semantic flaw?
- If self-checking catches 80% of structural violations before human review, does this improve or degrade human review quality? It might improve it (reviewers focus on harder problems) or degrade it (reviewers assume the pre-check caught everything and reduce scrutiny).

The answer becomes more nuanced when considering structured perspective diversity (Section

7.1). A single agent checking its own output is not meaningful validation — the same biases that produced the code will evaluate it. But a *structured ensemble of prompted perspectives* checking the generating agent's output is a different proposition: the reviewing agents share the generator's underlying model biases, but their prompted analytical frames constrain *what they attend to*, producing coverage across different subsets of the failure taxonomy (Appendix A). This is not independence, but it is orthogonality — the blind spots of a security-focused reviewer prompt and a data-quality-focused reviewer prompt overlap less than two identically-prompted agents' blind spots would. Whether this constitutes "validation" in a formal assurance sense is an open question, but it is a meaningfully stronger control than single-agent self-review.

9.6 Cross-Organisational Standards

Should there be a common agentic code security standard across Australian Government?

Individual organisations developing their own agentic code policies will produce inconsistent and potentially conflicting approaches. A common standard — even a lightweight one — would provide:

- A shared vocabulary for discussing agentic code risks (the taxonomy in Appendix A is a candidate starting point)
- A minimum bar for controls that all agencies using agentic coding must implement
- A basis for mutual recognition of agentic development practices across agencies
- Consistency in IRAP assessment criteria for agentic workflows

A transparency note is warranted here. The case study project described in Section 8 catches approximately 1–2 trust boundary violations per day at its CI gate. It does not currently share those patterns with other organisations. This paper recommends cross-organisational sharing of agent-introduced defect patterns, yet the project that generated the recommendation does not practise it. The reason is prosaic: no mechanism exists to make participation easy. There is no shared taxonomy for classifying these findings, no intake channel for reporting them, and no expectation that other organisations are looking for them. The gap this section identifies is not hypothetical — it is precisely the gap this paper's own practice falls into.

The counterargument: standardisation too early may lock in controls that prove inappropriate as the technology evolves rapidly. A vocabulary standard and minimum control set may be more durable than detailed prescriptive requirements.

9.7 The Correlated Failure Problem

How should risk models account for correlated failures in agent-generated code?

Traditional software risk models assume that defects are approximately independent — a bug in one function doesn't predict a bug in another. Agent-generated code violates this assumption (Section 2.4). A single training data bias produces the same failure mode across every function the agent generates.

This has implications for:

- **Testing strategy:** Independent sampling (testing a random subset of functions) underestimates defect rates when failures are correlated. If you find a trust boundary violation in one agent-generated function, the probability that the same violation exists in other agent-generated functions is much higher than if a human had written them.

- **Risk assessment:** The risk of a single agent-generated defect may be low, but the risk of a *systematic* defect affecting dozens or hundreds of functions is qualitatively different. How should risk registers capture correlated agent failure risk?
- **Remediation scope:** When a defect pattern is found in agent code, remediation should not be limited to the specific instance. The entire codebase should be scanned for the same pattern — because correlated failures mean the pattern is likely repeated.
- **Triage model:** Correlated failures mean 50 firings of the same rule across a codebase is one systematic issue requiring a systematic fix, not 50 independent tickets. Organisations that triage agent-generated defects as independent findings will overwhelm their remediation capacity on what is, operationally, a single root cause.

9.8 Contracted-Out Development

How does the threat model apply when most code is written by contractors, not agencies?

The majority of software development for Australian Government systems is performed by contracted service providers, not by agency staff. This paper's threat model applies to contractors' agents with the same force — an agent operated by Accenture, Leidos, or a boutique consultancy produces the same correlated, context-inappropriate patterns as an agent operated in-house. But the governance implications differ:

- **Visibility:** The contracting agency may have limited visibility into whether a contractor is using agentic tools, what proportion of deliverables are agent-generated, and whether the contractor's review processes address the failure modes in Appendix A. ISM-2074's AI usage policy requirement applies to the agency's own use; it is less clear how it flows down to contracted development.
- **Acceptance criteria:** Current delivery acceptance criteria for contracted software development typically focus on functional requirements, test coverage, and compliance with coding standards. They do not typically address the semantic correctness properties this paper identifies — trust boundary maintenance, audit trail integrity, context-appropriate error handling. A contractor could deliver code that meets every contractual requirement while containing systematic ACF-pattern violations.
- **Review responsibility:** When a contractor delivers agent-generated code, who is responsible for the security-focused review — the contractor's internal review process, the agency's acceptance review, or both? If the agency relies on the contractor's review, the agency inherits the contractor's review capacity constraints and habituation dynamics (Section 4.2).
- **Correlated risk across contracts:** If multiple agencies contract the same provider, and that provider uses the same agent tooling and prompts across engagements, the correlated failure problem (Section 9.7) extends across agency boundaries through the contractor, even if the agencies have no direct relationship.

This is arguably the most operationally significant gap for ASD to address, because it is where the majority of government code is actually produced. However, the contracted development assumption itself may be shifting. Agentic AI lowers the barrier to code production — not just for in-house development teams, but for the non-developer power users described in Section 1.2.8. If agencies begin generating more code internally, whether through technical staff using agents or through analysts and operators producing executable logic, the balance between contracted and in-house code may change. The governance challenge then compounds: agencies must address agentic code risks in both their contracted deliverables and their own expanding internal code production. (See Recommendation 6, §10.1, for candidate controls.)

10. Recommendations

The following recommendations are **candidate controls presented for community consultation**, not normative guidance. They reflect the authors' assessment of what is achievable and useful, grounded in the observed patterns and analytical framework in this paper. They are grouped by audience. For organisations seeking immediate priorities, three recommendations have the highest leverage and are achievable in the near term:

- **Recommendation 2** (§10.1, for policy bodies): *Issue guidance on treating agent output as a trust boundary.* This provides the conceptual foundation all other controls build on.
- **Recommendation 3** (§10.1, for policy bodies): *Extend ISM controls for agent-generated code.* The June 2025 ISM overhaul provides strong foundations that need only targeted extensions.
- **Recommendation 11** (§10.3, for organisations): *Document institutional security knowledge in machine-readable form.* This is the most direct defence against agents that lack institutional context, and requires no new tooling.

10.1 For Security Policy Bodies (ASD, ACSC)

1. **Develop a taxonomy of agentic code failure modes** based on established frameworks (STRIDE or equivalent). This paper's taxonomy (Appendix A) is a starting point, not a final product. The taxonomy should be developed in consultation with organisations that are already using agentic coding in assessed systems, to ground it in observed failure modes rather than theoretical concerns.

2. **Issue guidance on treating agent output as a trust boundary.** Clarify how agent-generated code should be classified in the trust model and what validation is required before integration into assessed systems. At minimum, guidance should address:

    - Whether agent-generated code requires different review criteria than human-authored code
    - Whether organisations must track which code was agent-generated (provenance)
    - What constitutes "sufficient validation" at the agent output boundary

3. **Extend ISM controls for software development** to address the distinct risk profile of agent-generated code. The June 2025 ISM overhaul added strong foundations (ISM-2060/2061 for code review, ISM-0402 for comprehensive testing, ISM-2026–2028 for artefact integrity), but these controls assume human-paced development. Candidate extensions:

    - ISM-0401 (Secure by Design): Require that Secure by Design practices include machine-enforceable rules when AI agents are used, since agents cannot internalise design principles from documentation.
    - ISM-2060/2061 (code review / security-focused peer review): Address review capacity scaling and the habituation effect under agent-volume code generation. Consider requiring measured review effectiveness, not just documented process.
    - ISM-0402 (testing with SAST/DAST/SCA): Extend SAST requirements to include semantic boundary analysis — trust tier flow verification, defensive pattern detection in audit-critical contexts — beyond current known-vulnerability-pattern scanning.
    - ISM-2026–2028 (artefact integrity): Extend supply chain controls to cover agent-generated first-party code as a supply chain input with distinct risk properties (correlated failures, no persistent learning).

- New control: Require organisations using agentic coding to demonstrate that their review and validation processes remain effective under the volume of agent-generated code they produce.

4. **Commission research into automated semantic boundary enforcement** as a complementary control to human review. The technical feasibility finding (Appendix B) suggests this is tractable — the core analysis uses only Python's standard library AST module and the verification properties are independently auditable (Section B.5). Research priorities:
   - Validate the feasibility finding against a broader set of codebases and programming languages
   - Evaluate whether automated enforcement meaningfully reduces defect escape rates in practice
   - Develop criteria for assessing the assurance properties of enforcement tools themselves (Appendix B, Section B.5)
5. **Extend SDLC-equivalent controls to executable logic produced outside formal development teams.** Agentic tools are expanding the population of software producers beyond professional developers (Section 1.2.8). Organisations should inventory and govern agent-generated plugins, automations, BI extensions, workflow scripts, low-code components, and similar artefacts produced by analysts, operators, and other non-developer staff. Where such artefacts affect trust boundaries, access control, audit trails, or data integrity, they should be subject to provenance, review, and validation controls proportionate to their impact — even when the producers do not consider themselves developers and the artefacts do not live in a formal version control system. The governance perimeter has expanded; controls must follow it.
6. **Address agent-generated code in contracted development.** The majority of software development for Australian Government systems is performed by contracted service providers (Section 9.8). Contracted developers' agents produce the same correlated, context-inappropriate patterns as in-house agents, but with additional governance challenges. Candidate controls:
   - Contract clauses requiring disclosure of agentic tool use and provenance tracking for agent-generated deliverables
   - Acceptance criteria that include semantic correctness properties (trust boundary maintenance, audit trail integrity, context-appropriate error handling), not just functional requirements and test coverage
   - Clarification of how ISM-2074's AI usage policy requirement flows down to contracted development
   - Assessment of correlated failure risk when the same contractor and agent tooling serves multiple agencies — a systematic defect introduced by a contractor's agent may propagate across agency boundaries through the shared provider

10.2 For IRAP Assessors

1. **Include agentic development practices in assessment scope** when organisations use AI coding agents in assessed systems. Evaluate:

- The validation boundary between agent output and code integration (Section 5.3)
- Evidence of review effectiveness under volume, not just existence of review process
- Provenance tracking for agent-generated code
- Whether the organisation has identified and addressed the failure modes relevant to their system's classification and data sensitivity

2. **Assess review quality, not just review process.** A documented code review process that is overwhelmed by volume provides less assurance than a smaller, more thorough review supported by automated pre-screening. Ask for evidence: defect escape rates, review depth audits, or automated pre-screening coverage metrics (Section 9.4).

3. **Consider correlated failure risk.** Traditional defect models assume independent failures. Agent-generated code produces correlated failures (Section 9.7). When assessing test coverage and defect rates, verify that the organisation's testing strategy accounts for correlation — finding one instance of a failure pattern should trigger codebase-wide scanning for the same pattern.

10.3 For Organisations Using Agentic Coding

1. **Treat agent-generated code as external input** requiring validation at the boundary. Do not assume agent output is correct because it passes tests and follows conventions. The failure modes in Appendix A are specifically designed to pass tests and look correct — that is what makes them dangerous.

2. **Document your institutional security knowledge in machine-readable form.** The gap between "what Python permits" and "what our system requires" is institutional knowledge that currently lives in documentation, team culture, and individual expertise. Encoding it in machine-checkable rules — whether through a purpose-built tool (Appendix B), project-specific linter rules, or structured review checklists — is the most direct defence against agents that don't share that knowledge.

3. **Invest in automated semantic boundary enforcement** as a complement to human review. The human review budget is finite; automated tools that handle structural trust boundary violations free human reviewers to focus on semantic issues that require institutional knowledge. The case study (Section 8.5) demonstrates that this is a redirection of existing effort, not additional overhead.

4. **Measure and monitor review effectiveness** under agentic volume. If review quality is degrading, address it — through tooling, capacity, or rate-limiting — before it becomes a compliance gap. Don't wait for an incident to discover that the review process is no longer providing the assurance it claims.

5. **Develop domain-specific review prompts aligned to your threat model.** Rather than relying on generic "review this code" instructions, create prompted analytical perspectives that map to specific failure categories — security boundary review for ACF-T1/E1, operational resilience review for ACF-R1/R2, data quality review for ACF-S1/S2. Critically, do not limit these to technical perspectives. Analytical frames like systems thinking (detecting feedback loops, shifted burdens, and second-order effects) and quality engineering (evaluating test strategy, coverage gaps, and verification adequacy) catch classes of defect that purely technical prompts miss — organisations will naturally think of the technical skills they need but overlook the logical and analytical ones. These prompts are reviewable, version-controlled security artefacts that encode institutional knowledge in a form complementary to the machine-readable rules described in Section 7.2 and Appendix B. Run

multiple prompted perspectives as a structured pre-screening step before human review to increase coverage breadth at agent speed (Section 7.1).

6. **When an agent-generated defect is found, scan for the pattern, not just the instance.** Correlated failures mean the same defect likely exists in other agent-generated code. Treating each defect as isolated underestimates the actual risk.
7. **Contribute to the community vocabulary.** Document your organisation's experience with agentic code failure modes and share (at an appropriate classification) to build the collective understanding. The taxonomy in this paper was developed from a single project's experience; it needs validation and extension from diverse government contexts.

Appendix A: Agentic Code Failure Taxonomy

A structured catalogue of failure modes, mapped to STRIDE categories, with detection characteristics, code examples, and risk ratings. Each entry includes the *reason agents produce this pattern* — understanding why helps calibrate both detection tools and review processes.

Summary Table

ID	Name	STRIDE	Risk	Existing Detection
ACF-S1	Competence spoofing	Spoofing	High	None
ACF-S2	Hallucinated field access	Spoofing	High	Partial
ACF-S3	Structural identity spoofing	Spoofing	High	Partial
ACF-T1	Trust tier conflation	Tampering	Critical	None
ACF-T2	Silent coercion	Tampering	Medium	None
ACF-R1	Audit trail destruction	Repudiation	High	Partial
ACF-R2	Partial completion	Repudiation	High	None
ACF-I1	Verbose error response	Info Disclosure	Medium	Partial
ACF-I2	Stack trace exposure	Info Disclosure	Low	Good
ACF-D1	Finding flood	DoS	High	N/A
ACF-D2	Review capacity exhaustion	DoS	High	N/A
ACF-E1	Implicit privilege grant	Elevation	Critical	None
ACF-E2	Unvalidated delegation	Elevation	High	Partial

Detailed Entries

Policy readers: the Summary Table above and the Detection Capability Summary at the end of this appendix provide a complete overview without requiring code fluency. The detailed entries below, which include Python code examples, are provided for technical practitioners, tool builders, and organisations implementing detection capabilities. Non-Python readers may rely on the Description, Why it's dangerous, and Detection approach fields for each entry.

Language specificity. The code examples throughout this appendix use Python, reflecting the case study environment. The failure modes vary in language-generality:

- **Language-general** (applicable across Python, Java, C#, TypeScript, Go, etc.): ACF-T1 (trust tier conflation), ACF-T2 (silent coercion), ACF-R1 (audit trail destruction), ACF-R2 (partial completion), ACF-I1 (verbose error response), ACF-D1 (finding flood), ACF-D2 (review capacity exhaustion), ACF-E1 (implicit privilege grant), ACF-E2 (unvalidated delegation). The failure *patterns* differ by language (e.g., `catch (Exception e)` in Java, `catch` in C++, `recover()` in Go), but the failure *mode* is the same.
- **Python-specific surface form** (same underlying failure, different manifestation in other languages): ACF-S1 (`.get()` with defaults — other languages have analogues like `Optional.orElse()` in Java or `??` in C#), ACF-S2 (`getattr()` with defaults — Python-specific, though dynamic languages like Ruby have `send`/`respond_to?`), ACF-S3 (`hasattr()` as capability gate — Python-specific surface form, though the underlying

failure applies to any language with duck typing or structural typing; Ruby's `respond_to?`, Go's interface satisfaction, and TypeScript's structural type compatibility are analogues), ACF-I2 (stack trace exposure — the specific mechanisms are framework-dependent everywhere).

Organisations working in other languages should read the *Description* and *Why it's dangerous* fields as language-general, and treat the *Example* and *Detection approach* fields as Python-specific reference implementations.

ACF-S1: Competence Spoofing

STRIDE: Spoofing | **Risk:** High | **Detection:** None

Description: Default values fabricate data where the absence of data should be surfaced as a failure, error, or explicit "unknown." The code presents a confident result that is actually based on fabricated input.

Why agents produce this: The `.get(key, default)` pattern appears in millions of Python files. In most contexts, providing a default for missing keys is genuinely good practice — a web application displaying "Unknown" for a missing user name is fine. Agents learn this as a universal pattern and apply it in contexts where the default fabricates safety-critical data.

Example:

```python
# Agent-generated - looks defensive and robust
def assess_risk_level(record):
    classification = record.get("security_classification", "OFFICIAL")
    clearance = record.get("required_clearance", "baseline")
    return classification, clearance

# Correct for high-assurance context - absence is a failure
def assess_risk_level(record):
    if "security_classification" not in record:
        raise MissingSecurityClassification(
            f"Record {record['id']}: security_classification absent - "
            f"upstream data integrity failure, cannot assess risk"
        )
    if "required_clearance" not in record:
        raise MissingSecurityClearance(
            f"Record {record['id']}: required_clearance absent - "
            f"cannot determine access level, refusing to default"
        )
    return record["security_classification"], record["required_clearance"]
```

Why it's dangerous: The first version silently downgrades security classifications when data is missing. A PROTECTED document with a corrupted or missing `security_classification` field is treated as OFFICIAL. Downstream access control decisions are based on the fabricated classification.

Detection approach: Flag `.get()` and `getattr()` with defaults on objects whose type is annotated with a trust tier of Tier 1 (internal/audit) or Tier 2 (validated pipeline data). Requires trust tier annotations (not available in existing tools).

ACF-S2: Hallucinated Field Access

STRIDE: Spoofing | **Risk:** High | **Detection:** Partial

Description: Agent accesses a field name that doesn't exist on the target object, masked by `getattr()` with a default. The code operates on fabricated data while appearing to access a real field.

Why agents produce this: Agents occasionally hallucinate field names — predicting a plausible field name that doesn't exist in the actual schema. Without `getattr`, this produces an immediate `AttributeError`. With `getattr(obj, "hallucinated_field", None)`, the error is silently suppressed and the code operates on `None` (or whatever default is provided).

Example:

```
# Agent hallucinated "risk_score" - actual field is "risk_rating"
threshold = getattr(assessment, "risk_score", 0)
if threshold > 5:
    escalate(assessment)
# risk_score is always 0 (the default), so nothing is ever escalated.
# The code looks correct. Tests pass (they test the escalation path with explicit values).
# The bug is invisible until someone notices that escalation never triggers.
```

Why it's dangerous: The code silently does nothing instead of crashing. In a security context, "nothing happens" can mean "threats are not escalated" or "alerts are not raised" — failures of omission that are harder to detect than failures of commission.

Detection approach: Type checkers (mypy, pyright) catch this *if the object is fully annotated*. If the object is `Any` or untyped, type checkers are silent. The semantic boundary enforcer (Appendix B) adds a complementary rule: `getattr` with a default on any object that has a declared type annotation is flagged, because the annotation means the field set is known and access should be direct.

ACF-S3: Structural Identity Spoofing

STRIDE: Spoofing (+ Elevation of Privilege consequence) | **Risk:** High | **Detection:** Partial

Description: A `hasattr()` check is used as a capability or privilege gate, allowing any object that declares the expected attribute to pass — regardless of whether the object is of the correct type. The gate accepts structural presence as proof of identity.

Why agents produce this: `hasattr()` is the idiomatic Python pattern for duck-typing capability checks. Training data is saturated with it — agents building plugin systems, authorisation checks, or capability dispatchers will reach for `hasattr` by default because it is the "Pythonic" way to test whether an object supports an operation. The concept that structural presence is not ontological identity — that *having* an attribute is not the same as *being* the right type — is a security distinction that the language actively discourages.

Example:

```
# Agent-generated - "Pythonic" duck-typing capability check
def process_classified(obj):
    if hasattr(obj, "security_clearance"):
        handle_classified(obj)  # Any object with this attr gets in

# Trivial bypass - no type hierarchy modification needed
class Impersonator:
    security_clearance = "TOP_SECRET"  # Just declare the attribute

process_classified(Impersonator())  # Gate opens
```

```
# Correct - requires actual type membership
def process_classified(obj):
    if isinstance(obj, ClearedPersonnel):
        handle_classified(obj)  # Must inherit from ClearedPersonnel
    # Cannot bypass without modifying the class hierarchy itself
```

Why it's dangerous: Unlike ACF-S1 (data fabrication via defaults) where the fabricated value is visible at the call site, the exploit surface for `hasattr` gates is anywhere an object is constructed — potentially far from the gate. The gate looks secure in isolation. Worse, Python's `__getattr__` protocol means a single class can dynamically claim to possess *any* attribute:

```
class UniversalImpersonator:
    def __getattr__(self, name):
        return True  # "Yes, I have that. And everything else."

# This object passes EVERY hasattr check in the entire codebase.
# An isinstance check is immune to this.
```

This is the capability-based equivalent of ACF-S1's competence spoofing: ACF-S1 fabricates *data* where absence should be a failure; ACF-S3 fabricates *identity* where type membership should be required. The object claims to be something it isn't, and the gate believes it because the check is structural (has the attribute) rather than ontological (is the type). The elevation of privilege consequence follows directly — the impersonator passes through a privilege gate that should have rejected it.

Detection approach: An unconditional lint rule banning `hasattr()` catches all instances (the case study codebase in Section 8 enforces this). General-purpose linters do not flag `hasattr` because it is considered idiomatic Python. The semantic boundary enforcer (Appendix B) treats `hasattr` as unconditionally prohibited — unlike `.get()` or `getattr()`, which are context-dependent, there is no legitimate use of `hasattr` that cannot be expressed more safely as `isinstance()`, explicit `try/except AttributeError`, or an allowset check. Detection is rated Partial because the rule is simple to implement but not present in any widely-deployed tool.

ACF-T1: Trust Tier Conflation

STRIDE: Tampering | **Risk:** Critical | **Detection:** None

Description: Data from an external (untrusted) source is used in an internal (trusted) context without passing through a validation boundary. The data's effective trust level is silently elevated.

Why agents produce this: Python's type system doesn't distinguish between data from different sources. A `dict` from `requests.get().json()` and a `dict` from a validated internal query are the same type. Agents see both as "a dict" and treat them interchangeably because nothing in the language tells them otherwise.

Example:

```
# Agent-generated - clean, readable, catastrophically wrong
def sync_partner_records(partner_api_url):
    response = requests.get(f"{partner_api_url}/records")
    records = response.json()
    for record in records:
        db.execute(
            insert(internal_records).values(**record)
        )
```

```
    # External data inserted directly into internal database.
    # No schema validation, no field allowlisting, no type checking.
    # Partner could send arbitrary fields, wrong types, injection payloads.

# Correct - validate at the boundary
def sync_partner_records(partner_api_url):
    response = requests.get(f"{partner_api_url}/records")
    raw_records = response.json()
    for raw in raw_records:
        try:
            validated = PartnerRecordSchema.validate(raw)
        except ValidationError as e:
            quarantine(raw, reason=str(e))
            continue
        db.execute(
            insert(internal_records).values(
                name=validated.name,
                status=validated.status,
            )
        )
```

Why it's dangerous: This is the most critical failure mode because it compromises the integrity of the internal data store — the system's source of truth. Once external data enters the internal store without validation, every downstream consumer trusts it as internal data. Corruption propagates invisibly.

Detection approach: Taint analysis — trace the return values of functions marked `@external_boundary` (or matched by the known external call heuristic list) and flag if they reach data store operations without passing through a function marked `@validates_external`. This is the core capability of the tool described in Appendix B.

ACF-T2: Silent Coercion

STRIDE: Tampering | **Risk:** Medium | **Detection:** None

Description: Type coercion across trust boundaries hides data quality issues. Values are silently converted to a compatible type rather than being flagged as invalid.

Why agents produce this: Python's `or` operator and conditional expressions make coercion easy and idiomatic. `value = input_value or "default"` is a common pattern. Agents apply it broadly without distinguishing between contexts where coercion is appropriate (Tier 3 → Tier 2 at a validated boundary) and contexts where it's dangerous (Tier 1 internal data that should never need coercion).

Example:

```
# Silent coercion hides data quality problem
amount = float(row.get("transaction_amount", 0))
# Missing transaction amount is silently zero - not "unknown" or "error."
# A zero-value transaction passes every downstream check.
# An audit query for "transactions over $1000" won't find it,
# but neither will "transactions with missing amounts."

# Better - make the absence explicit
if "transaction_amount" not in row:
```

```
    return TransformResult.error({"reason": "missing_amount", "row_id": row_id})
amount = float(row["transaction_amount"])
```

Detection approach: Flag coercion patterns (`.get()` with non-None defaults, `or` chains with fallback values, ternary expressions with defaults) on fields from Tier 1 or Tier 2 data. The distinction from ACF-S1 is that this involves type conversion, not just default substitution.

ACF-R1: Audit Trail Destruction

STRIDE: Repudiation | **Risk:** High | **Detection:** Partial

Description: Broad exception handlers catch errors from audit-critical operations and log-and-continue rather than propagating the failure to the audit system.

Why agents produce this: "Catch exceptions and log them" is a pervasive pattern in training data. In most applications, it's reasonable — a web server should log errors and keep serving. Agents apply this pattern to audit-critical operations without recognising that some failures must propagate rather than be absorbed.

Example:

```
# Agent-generated - looks like responsible error handling
try:
    record_decision(case_id, decision, rationale, evidence)
except Exception as e:
    logger.error(f"Failed to record decision for {case_id}: {e}")
    # Decision was made. Decision was not recorded.
    # The audit trail now has a gap that cannot be reconstructed.
    # The log message may be rotated away. The decision stands, unrecorded.
```

Why it's dangerous: In regulatory contexts, the audit trail is the legal record. A gap in the audit trail is not just a logging failure — it's a compliance failure that may have legal consequences. "We made a decision but can't prove what it was based on" is an unacceptable answer in a formal inquiry.

Detection approach: Existing linters flag bare `except:` (no exception type) but not `except Exception:` (which is considered acceptable practice). Semantic detection requires understanding which operations are audit-critical — this is project-specific knowledge encoded in the trust topology (e.g., functions annotated as audit-write operations should not be inside broad exception handlers that continue on failure).

ACF-R2: Partial Completion

STRIDE: Repudiation | **Risk:** High | **Detection:** None

Description: A sequence of operations that should be atomic (all-or-nothing) is implemented without rollback, so partial failure leaves the system in an inconsistent state.

Why agents produce this: Agents implement operations sequentially and add error handling per-step. They don't naturally recognise that a group of operations should be treated as a transaction unless explicitly prompted. The concept of "these three operations must all succeed or all fail" is a design decision, not a language feature.

Example:

```
# Agent-generated - each step has error handling, but no atomicity
def reclassify_document(doc_id, new_classification):
    update_classification(doc_id, new_classification)  # Step 1: succeeds
    notify_stakeholders(doc_id, new_classification)     # Step 2: fails (network error)
    record_reclassification(doc_id, old, new)           # Step 3: never runs
    # Document is reclassified, stakeholders don't know, audit trail is incomplete.
    # If step 2 is wrapped in try/except and continues, step 3 records a
    # reclassification that stakeholders were never notified about.
```

Detection approach: No existing tool detects this — it requires understanding which operations form a logical transaction. A semantic boundary enforcer could flag functions that contain multiple audit-write operations without a transaction context, but this requires project-specific annotation of which operations are audit-critical.

ACF-I1: Verbose Error Response

STRIDE: Information Disclosure | **Risk:** Medium | **Detection:** Partial

Description: Error handlers expose internal system details (database schemas, file paths, query parameters, library versions) in error responses.

Why agents produce this: Agents produce "helpful" error messages that include full context. During development, this is valuable. In production, it's reconnaissance information. Agents don't distinguish between development and production error handling because the distinction is contextual, not syntactic.

Example:

```
except DatabaseError as e:
    return {"error": str(e), "query": sql, "connection": str(db_url)}
```

Detection approach: Existing scanners detect some cases (credential patterns, known sensitive variable names). Comprehensive detection requires understanding which variables contain sensitive information — a context-dependent judgment.

ACF-I2: Stack Trace Exposure

STRIDE: Information Disclosure | **Risk:** Low | **Detection:** Good

Note: This entry is included for taxonomy completeness. It is well-covered by existing tooling and is lower risk than other entries. It could reasonably be treated as a sub-variant of ACF-I1.

Description: Full Python tracebacks returned in API responses or user-facing error messages.

Why agents produce this: `traceback.format_exc()` is a common debugging pattern. Agents include it in error handlers without considering the deployment context.

Detection approach: Well-covered by existing tools. Most security scanners and web framework linters detect traceback exposure.

ACF-D1: Finding Flood

STRIDE: Denial of Service | **Risk:** High | **Detection:** N/A (process threat)

Description: The volume of static analysis findings on agent-generated code overwhelms reviewers, causing them to rubber-stamp findings without evaluation.

Why this happens: Agents produce code at volume, and if that code triggers many findings, the review queue grows faster than the review capacity. Reviewers under volume pressure shift from evaluating each finding to batch-dismissing them. The DoS is against the *review process*, not the system.

Mitigation: Finding caps per rule per file (Appendix B), prioritised finding presentation (critical first), and measured suppression rates as a health metric.

ACF-D2: Review Capacity Exhaustion

STRIDE: Denial of Service | **Risk:** High | **Detection:** N/A (process threat)

Description: Agent code generation velocity exceeds the organisation's capacity for security-focused review, degrading review from active verification to passive scanning.

Why this happens: Agents can generate plausible, convention-conforming code faster than review processes were designed to absorb (Section 1.2.1). Review capacity doesn't scale at the same rate. The review process becomes a bottleneck, and the organisational response is often to lower the review bar rather than reduce the generation rate.

Mitigation: Automated pre-screening to reduce the human review burden, volume-aware capacity planning, and measured review effectiveness metrics (Section 9.4).

ACF-E1: Implicit Privilege Grant

STRIDE: Elevation of Privilege | **Risk:** Critical | **Detection:** None

Description: External system assertions are accepted without independent verification, granting privileges based on unvalidated claims.

Why agents produce this: Agents implement integration patterns by calling external APIs and acting on the response. The concept that the external system's response must be independently verified — that the response itself is untrusted — is not visible in the code structure. The code looks like a normal API call and response handling.

Example:

```
partner_verification = partner_api.verify_identity(applicant_id)
if partner_verification.get("verified", False):
    grant_system_access(applicant_id, level="standard")
# Partner says "verified" → access granted.
# No independent check. No recording of the basis for the decision.
# If the partner system is compromised, every applicant is "verified."
```

Detection approach: Taint analysis — the return value of an `@external_boundary` function is used as a predicate in an access control decision without passing through validation. Requires both boundary annotation and understanding of which operations are access-control-relevant.

Related: ACF-S3 (structural identity spoofing) describes a complementary elevation mechanism where the gate itself is structurally unsound — `hasattr()` gates accept any object that declares the expected attribute, regardless of type. ACF-E1 is about *unvalidated external assertions*; ACF-S3 is about *unsound gate predicates*. Both result in implicit privilege grants, but through different mechanisms.

ACF-E2: Unvalidated Delegation

STRIDE: Elevation of Privilege | **Risk:** High | **Detection:** Partial

Description: User-supplied parameters are used directly in privileged operations (database queries, file access, system commands) without validation or restriction.

Why agents produce this: The pattern `db.query(Model).filter_by(**user_params)` is concise and idiomatic. Agents produce it because it's the shortest path from input to query. The concept that user parameters must be restricted to an allowlist of permitted fields is a security requirement, not a language requirement.

Example:

```
def search_records(user_query: dict):
    return db.query(Record).filter_by(**user_query)
    # User can filter on internal fields: is_deleted, internal_score,
    # admin_notes - fields that should not be queryable.
```

Detection approach: SQL injection scanners catch some cases (especially string interpolation into SQL). Parameter delegation via `**kwargs` unpacking into ORM queries is less consistently detected. Semantic detection requires understanding which operations are privileged and which parameters are user-controlled.

Detection Capability Summary

Detection Level	Count	Failure IDs	Implication
None (no existing tool detects it)	5	ACF-S1, ACF-T1, ACF-T2, ACF-R2, ACF-E1	These require new tooling or new review practices
Partial (some tools catch some cases)	5	ACF-S2, ACF-S3, ACF-R1, ACF-I1, ACF-E2	Existing tools provide incomplete coverage; augmentation needed
Good (existing tools generally catch it)	1	ACF-I2	Already addressed by current tooling
N/A (process threat, not code pattern)	2	ACF-D1, ACF-D2	Requires process controls, not technical controls

This distribution — 5 of 13 failure modes completely undetectable by existing tools, and 5 more only partially detected — is the gap this paper identifies. The 5 undetectable failure modes include both Critical-rated entries (ACF-T1, ACF-E1), meaning the highest-risk failures are precisely the ones that current tooling misses entirely.

Appendix B: Technical Feasibility of Automated Enforcement

B.1 Design History

The threat model in this paper identifies failure modes that existing tools don't detect (Appendix A: 5 of 13 failure modes have no existing tool coverage). The natural question is: can these be detected automatically?

The design presented in this appendix emerged through three iterations of structured adversarial deliberation using prompted AI agent teams, spanning several days. The iterative history is documented here because it is itself evidence for the paper's thesis: agent output treated as external input (Section 5.2), validated through structured review (Section 7.1), with the human providing framing and acceptance judgment rather than technical content.

Iteration 1: "Can we fix Python?" The initial framing asked whether Python's permissive defaults could be addressed at the language level — a stricter dialect, a transpiler, or a runtime enforcement layer. An agent team deliberated across five rounds and concluded unanimously that the approach was not viable. The ecosystem orphaning risk (a Python variant diverges from CPython with every release), the adoption cliff, and the maintenance burden were each independently fatal.

Iteration 2: "What can we do instead?" The human contribution at this stage was a question, not a direction. A second agent team — operating with knowledge of the first team's conclusions but no other constraint — independently generated the concept that became the semantic boundary enforcer: a standalone AST-based analysis layer that extends Python's existing annotation machinery rather than creating a parallel system. This team conducted eight rounds of structured adversarial deliberation and produced the initial design specification. The design was implemented as a pattern-matching enforcement gate and deployed in production on the case study codebase (Section 8), where it catches the 1–2 violations per day reported in Section 8.4.

The creative pivot — from "change the language" to "analyse the language" — was generated by the agent team, not by the human operator. The human asked an open question; the agents produced the core insight that the problems identified in Iteration 1 are analysable over standard Python using its existing annotation system. The human contribution was the question, the validation structure, and the judgment to proceed — not the design concept itself.

That existing tool operates by matching known defensive anti-patterns (`.get()` with defaults on typed data, broad `except` clauses on audit-critical paths, `hasattr()` usage) against a per-module allowlist. It is effective for the patterns it recognises, but it has structural limitations: it cannot trace data flow across function boundaries, it cannot verify that a function decorated as a validator actually contains control flow, and it cannot distinguish contextually appropriate uses of a pattern from dangerous ones without per-finding allowlist entries.

Iteration 3: "What does the best version look like?" The third iteration was designed as a reproducibility test — a clean-start roundtable with the same problem statement. It used seven specialist agent perspectives (security architect, Python AST engineer, systems thinker, adversarial red-teamer, quality engineer, governance designer, integration engineer) with a dedicated scribe agent maintaining shared working memory between rounds. The scribe had authority to challenge agents who contradicted their prior stated positions — functioning as a committee chair enforcing epistemic accountability.

The process included steelman rounds (each agent argues for the *strongest* version of the

position they most disagree with) and structured challenge rounds (independent fault-finding). These mechanisms produced measurably different results from single-pass review: binary taint tracking was rejected 7/7 after cross-agent challenge exposed it as a compliance ritual; runtime trust tagging was rejected after four of seven agents independently converged on the same fatal objection; and the team unanimously rejected the human-agent dual enforcement model from Iteration 2 on grounds that authorship attribution is unsolvable in mixed workflows — an argument the original roundtable never surfaced.

The reproducibility finding was nuanced: the process dynamics reproduced (convergent rejections, forced concessions), but specific architectural decisions diverged. Most significantly, the team replaced binary taint with a two-dimensional model tracking provenance (where data came from) and validation status (what processing it received) as orthogonal dimensions — an insight two agents independently converged on in parallel, which constitutes stronger evidence for its validity than a single agent proposing it. The design presented in this appendix reflects the third iteration's architecture, noting where it extends the production implementation.

The human role across iterations. In Iteration 1, the human posed the initial question. In Iteration 2, the human posed the follow-up question, validated the agent-generated concept, and judged when the design was sufficient for implementation. In Iteration 3, the human designed the reproducibility experiment and will ultimately judge whether the more sophisticated design warrants implementation. At no stage did the human generate the core technical insight — but at every stage, the human determined which agent outputs entered the trusted store. This is the paper's proposed workflow (Section 5.3) operating at the design level.

Feasibility finding. The existing production deployment demonstrates that automated detection of the most critical agentic code failure modes is technically feasible for Python, buildable at modest cost, and compatible with existing development workflows — even with the relatively simple pattern-matching approach of Iteration 2. The enhanced design presented here extends that foundation with provenance-aware taint tracing and structural verification, addressing the limitations above. It is presented so that other organisations can build equivalent tooling for their own codebases and languages — whether the approach generalises beyond Python is an open question (see Recommendation 4). The full roundtable synthesis — including the complete rule evaluation matrix, governance model, and minority reports — is available as a companion document.

B.2 Core Properties

Property	Detail
Dependencies	Zero — Python standard library `ast` module only
Language compatibility	Analyses standard Python; no custom syntax or runtime modifications
Output format	SARIF (Static Analysis Results Interchange Format)
Delivery	Standalone PyPI package

Property	Detail
Taint model	Two-dimensional: provenance (where data came from) × validation status (what processing it received) — 7 effective states determining finding severity
Analysis scope	Intra-function taint analysis (v0.1); inter-procedural deferred to v1.0

Note: the relevant measure of this tool is not its implementation size but its **verification properties** — see Section B.5.

B.3 How It Works

Declaration. Developers declare trust boundaries using standard Python annotations and decorators — no custom syntax, no new imports beyond the tool's own library:

```
@external_boundary          # Returns untrusted data
def fetch_api_data(): ...

@validates_external         # Validates external data (must contain control flow)
def validate_response(): ...
```

A built-in heuristic list recognises common external call sites (`requests.*`, `httpx.*`, `sqlalchemy.*.execute`, `json.loads`, etc.) without requiring explicit annotation — reducing the annotation burden for common patterns while allowing project-specific declarations for uncommon ones.

Two-dimensional taint model. Unlike binary taint tracking (tainted/clean), the tool tracks two independent dimensions for every variable in scope. *Provenance* — where the data came from — is immutable once assigned: `TIER_1` (audit trail), `TIER_2` (pipeline data), `TIER_3` (external), `UNKNOWN`, or `MIXED` (container holding values from multiple tiers). *Validation status* — what processing the data received — monotonically increases from `RAW` to `STRUCTURALLY_VALIDATED`. Validation is only meaningful for `TIER_3` and `UNKNOWN` data; internal data does not pass through `@validates_external`. This reduces the theoretical 5×2 matrix to 7 effective states, each producing different finding severity for the same pattern. For example, `.get()` with a default on `TIER_1` data is an unconditional error (audit trail fabrication), on `TIER_2` data is a standard error (contract violation), and on validated `TIER_3` data is suppressed (the validation boundary already handled it).

Analysis. The tool performs two-pass analysis:

1. **Symbol collection:** Walks the AST to build a table of annotated boundaries, decorated functions, and known external call sites
2. **Rule evaluation:** Seven rules detect defensive programming patterns that are dangerous in high-assurance contexts — `.get()` with defaults, `getattr()` with fallbacks, `hasattr()` gates, broad `except` blocks, silent exception swallowing, unvalidated data reaching audit writes, and `isinstance()` guards on internal data. Each rule is evaluated against the 7 effective states, producing a 49-cell severity matrix where every combination of pattern and provenance has a defined outcome (error, warning, advisory, or suppress). The full matrix is documented in the companion roundtable synthesis.

Structural verification. Functions decorated with `@validates_external` must contain a rejection path in their body — reachable control flow that can reject invalid data (`try/except`, `isinstance()` with conditional, `raise`, or `if/else`). A validator without a rejection path is structurally unsound — it attests to validation without the ability to reject. This prevents agents from satisfying the taint checker by applying a decorator to a pass-through function.

B.4 Governance Model

The tool addresses the agent circumvention problem (Section 7 of the threat model) through three mechanisms, all of which are implemented and enforced in the production deployment described in Section 8:

Project-level manifest. A `strict.toml` file declares trust topology, boundary functions, rule configuration, and structured exceptions. Exceptions require a rationale, reviewer identity, and expiry date.

Human-gated governance. The manifest is protected by repository CODEOWNERS. Ratification of new exceptions must occur in a **separate, prior commit** from the source code changes — CI verifies the temporal separation via git commit history. This prevents agents from simultaneously generating a trust boundary violation and its exception in the same commit.

Exceptionability classification. Each cell in the 49-cell severity matrix is assigned to one of four governance classes: *UNCONDITIONAL* (no exceptions permitted — the tool rejects creation at parse time), *STANDARD* (grouped rationale with 90-day expiry and divergence detection), *LIBERAL* (single-line rationale with 180-day expiry), or *TRANSPARENT* (advisory or suppressed — below governance threshold). Twenty-four of 49 cells (49%) are UNCONDITIONAL — project invariants hardcoded in the tool's source, not in editable configuration. These include all `hasattr()` findings regardless of provenance, `.get()` on audit trail data, and broad exception handlers on audit-critical paths. An earlier iteration of the design used differential enforcement profiles for human-authored and agent-authored code; this was unanimously rejected on the grounds that authorship attribution is unsolvable in mixed workflows and that differential profiles create incentives to launder agent code through human commits.

B.5 Verification Properties (The Assurance Argument)

For a policy audience, the relevant question about this tool is not "how big is it?" but **"how do you know it's correct?"** This is especially pertinent if the tool is itself built with agent assistance — the tool's own development is subject to the threat model it exists to address.

The design includes four independent verification mechanisms:

1. Golden corpus. A collection of labelled Python snippets — true positives (code that should trigger findings) and true negatives (code that should not) — plus adversarial evasion samples (code that looks compliant but isn't). Minimum: 3 true positives + 2 true negatives per rule. The corpus is a first-class artefact, version-controlled, and a ship gate.

2. Self-hosting gate. The tool's own source code must pass its own rules in CI from the first commit. If the tool cannot be written to its own standards, the standards are not understood.

3. Measured precision with volume floor. Rules track their true positive rate across runs. An immutable 80% precision floor is hardcoded in the tool — below this, a rule cannot earn blocking status regardless of configuration. Per-rule thresholds are earned above this floor through corpus evidence (e.g., `hasattr()` at ~99%, `.get()` at ~88%) and are monotonically

non-decreasing: once a rule earns a higher threshold, it cannot be lowered. This prevents both promotion based on small samples and regression of established precision.

4. Deterministic output. Identical input must produce byte-identical output. Verified by running the tool twice and diffing. Non-determinism in a security tool is a defect.

These properties are **independently evaluable by an assessor.** An IRAP assessor (or equivalent) can verify:

- The corpus exists and covers the claimed rules
- The self-hosting gate is enforced in CI
- Precision tracking data is available and shows the claimed rates
- Output is deterministic

This is a stronger assurance argument than "the implementation is N lines" or "the development took M hours." It answers the question *"how do you know the tool catches what it claims to catch?"* with measurable, auditable evidence.

B.6 Relationship to Existing Tools

Existing Tool	What It Does	What It Misses	Relationship
mypy/pyright	Verifies type shape	Data provenance (a `str` from an API and a `str` from a DB are the same type)	Complementary — this tool verifies provenance
ruff/flake8	Enforces style/idiom	Trust boundary semantics	Complementary — this tool handles trust boundaries; style belongs in ruff
bandit	Detects known vulnerability patterns	Project-specific trust violations	Complementary — bandit is generic; this tool is project-configured
semgrep	Custom pattern matching with taint analysis	Provenance-aware severity grading, exceptionability classification, manifest-based exception governance	Closest existing tool — but lacks the governance model

Why not extend an existing tool? The governance model (exceptionability classification with UNCONDITIONAL cells hardcoded in source, provenance-aware severity grading, temporal separation of manifest changes) is the differentiating capability. These are not features that can be added to existing tools as plugins — they require control over the finding lifecycle, which existing tools' architectures don't expose. The analysis engine is deliberately simple; the governance model is the novel contribution.

B.7 The Meta-Observation

If this tool is built with agent assistance — which is the natural development approach given the domain — its own development becomes the first test case for its thesis. If the design specification is tight enough that an agent can implement it, and the self-hosting gate and golden corpus catch the problems the agent introduces, the tool has demonstrated its value proposition through its own creation.

This recursive property is unusual for security tools and is worth noting for the policy audience: **the tool's development process is itself evidence for or against its claims.** An assessor can evaluate not just the tool's output but the conditions under which it was built — did the self-hosting gate catch agent-generated defects? Did the golden corpus reveal false positives or negatives? The development history is part of the assurance argument.

The full design specification is available as a companion document: *Semantic Boundary Enforcer — Design Specification* (forthcoming).

Appendix C: Agent Autonomy Self-Assessment

This appendix provides an informal diagnostic for organisations to identify where their current agent usage sits on the autonomy spectrum and whether their controls are proportionate. It is not a maturity model — higher tiers are not aspirational targets, and there is no implied progression from lower to higher. Most organisations will find themselves at different tiers simultaneously: Tier 1 for security-critical components, Tier 3 for test scaffolding, Tier 2 for general feature work. That is entirely appropriate, provided the controls at each tier match the risk profile described below.

The purpose is self-location. An organisation that discovers it is operating at Tier 3 without the controls listed for Tier 3 has identified a gap. An organisation operating at Tier 1 with controls designed for Tier 3 has identified waste. Neither outcome requires changing the tier — only aligning controls to reality.

	Tier 0: Full Human	**Tier 1: Prompted + Copied**	**Tier 2: IDE-Integrated**	**Tier 3: Autonomous**
What it looks like	No agent involvement. Human writes all code.	Developer asks agent specific questions, copies and adapts fragments into codebase manually.	Agent autocompletes functions and classes inline. Developer accepts or rejects suggestions in-editor.	Agent plans, implements, tests, and commits with minimal human intervention.
Who holds architectural context	Human	Human	Shared — human directs, agent infers from surrounding code	Agent — from project documentation, system prompts, and codebase patterns
Error correlation	Independent (human variation)	Low — fragments are isolated, human integrates	Moderate — agent infers patterns from local context and may replicate across completions	High — same training biases applied systematically across features (Section 2.4)
Review surface	Normal	Slightly elevated — more code to review, but each fragment is small	Elevated — easy to accept completions without full evaluation	Massive — entire features arrive at review boundary as finished artefacts (Section 4.1)

	Tier 0: Full Human	Tier 1: Prompted + Copied	Tier 2: IDE-Integrated	Tier 3: Autonomous
Habituation risk	Baseline	Low	Moderate — “tab-accept” becomes reflexive (Section 4.2)	High — output volume degrades review from verification to scanning
Minimum controls	Existing SDLC	Existing SDLC is likely adequate	Awareness of ACF patterns (Appendix A); SAST augmentation advisable	Validation boundary (Section 5.3), semantic enforcement (Section 7.2), provenance tracking (Section 7.3), measured review effectiveness (Section 9.4)

References

- Australian Signals Directorate. *Information Security Manual.* Commonwealth of Australia. December 2025 revision. https://www.cyber.gov.au/resources-business-and-government/essential-cyber-security/ism — Controls referenced: ISM-0401 (Secure by Design), ISM-0402 (SAST/DAST/SCA), ISM-1419 (development environments), ISM-2026/2027/2028 (software artefact integrity), ISM-2060 (code review), ISM-2061 (security-focused peer review), ISM-2074 (AI usage policy). Individual controls are searchable by number on the ASD website.
- Australian Signals Directorate. *Essential Eight Maturity Model.* Commonwealth of Australia. Updated periodically. https://www.cyber.gov.au/resources-business-and-government/essential-cyber-security/essential-eight
- NIST. *SP 800-218: Secure Software Development Framework (SSDF), Version 1.1.* February 2022. https://csrc.nist.gov/publications/detail/sp/800-218/final
- NIST. *SP 800-218A: Secure Software Development Practices for Generative AI and Dual-Use Foundation Models.* July 2024. https://csrc.nist.gov/publications/detail/sp/800-218a/final — (AI-specific SSDF supplement referenced in Section 6.2)
- Microsoft. *The STRIDE Threat Model.* Microsoft Security Development Lifecycle. https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats
- OASIS. *Static Analysis Results Interchange Format (SARIF), Version 2.1.0.* March 2020. https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html — (Output format referenced in Appendix B)
- OWASP. *Top 10 for LLM Applications.* 2025. https://owasp.org/www-project-top-10-for-large-language-model-applications/
- OWASP. *Secure Coding Practices — Quick Reference Guide.* 2010. https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/
- MITRE. *Common Weakness Enumeration (CWE).* The MITRE Corporation. https://cwe.mitre.org/ — (Taxonomy comparison in Section 6.4)
- Meadows, D. *Thinking in Systems: A Primer.* Chelsea Green Publishing, 2008. (Systems archetypes referenced in Section 4.2)
- Parasuraman, R. & Manzey, D. "Complacency and Bias in Human Use of Automation: An Attentional Integration." *Human Factors*, 52(3), 381-410, 2010. (Automation bias referenced in Section 4.2)
- Perry, N. et al. "Do Users Write More Insecure Code with AI Assistants?" *ACM CCS*, 2023. (AI-assisted developers wrote less secure code while feeling more confident — Section 4.2)
- Peng, S. et al. "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot." 2023. (Controlled study: 55.8% faster task completion — Section 1.2.1)
- Cui, K. et al. "The Effects of Generative AI on High Skilled Work: Evidence from Three Field Experiments with Software Developers." MIT/Microsoft/Accenture, 2024. (Field experiment: 12.9–21.8% more PRs/week at Microsoft — Section 1.2.1)
- Dutta, P. et al. "ASTRIDE: A Security Threat Modeling Platform for Agentic-AI Applications." *arXiv:2512.04785*, December 2025. https://arxiv.org/abs/2512.04785 — (STRIDE extension for AI agent-specific attacks, referenced in Section 3.1)
- Meier, J.D. et al. "STRIDE-LM Threat Model." *CSF Tools*. https://csf.tools/reference/stride-lm/ — (STRIDE extension adding Lateral Movement, referenced in Section 3.1)
- METR. "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity." July 2025. (RCT: experienced developers 19% slower with AI despite believing 20% faster — Sections 1.2.1, 4.2)

- Pichai, S. Alphabet Inc. Q3 2024 Earnings Call, 29 October 2024. "More than a quarter of all new code at Google is generated by AI, then reviewed and accepted by engineers." Reported in *The Verge*, 29 October 2024. https://www.theverge.com/2024/10/29/24282757/google-new-code-generated-ai-q3-2024 — (Section 1.2.1)
- Wolf, A. "Welcome to the Eternal September of open source. Here's what we plan to do for maintainers." *GitHub Blog*, 12 February 2026 (updated 13 February 2026). https://github.blog/open-source/maintainers/welcome-to-the-eternal-september-of-open-source-heres-what-we-plan-to-do-for-maintainers/ — Primary source. GitHub's Director of Open Source Programs on the review capacity crisis: "The cost to create has dropped but the cost to review has not." Announces PR access controls, interaction limits, and automated triage. (Referenced in Sections 1.2.2 and 4.1)
- Ghoshal, A. "GitHub eyes restrictions on pull requests to rein in AI-based code deluge on maintainers." *InfoWorld*, 4 February 2026. https://www.infoworld.com/article/4127156/github-eyes-restrictions-on-pull-requests-to-rein-in-ai-based-code-deluge-on-maintainers.html — Secondary reporting on the same events. GitHub described the problem as a denial-of-service attack on human attention. (Review capacity exhaustion evidence referenced in Sections 1.2.2 and 4.1)
- Graham-Cumming, J. "Incident report on memory leak caused by Cloudflare parser bug." Cloudflare Blog, 23 February 2017. https://blog.cloudflare.com/incident-report-on-memory-leak-caused-by-cloudflare-parser-bug/ — (Precedent referenced in Section 1.2)

Suggested citation: Morrissey, J. (ORCID: 0009-0000-5654-3782). "When Agents Write Code: A Threat Model for AI-Assisted Software Development in Government Systems." Discussion Paper, DRAFT v0.2, 8 March 2026. Digital Transformation Agency.